

PRINT-FLOW-360 · RESEARCH LIBRARY

Agent Orchestration

9 research documents · each with plain-language key takeaways

Compiled 2026-06-26

Table of Contents

1.	Agent Orchestration & Multi-Agent Systems — Suite Index
2.	Foundations: Agents, Agentic Loops & When to go Multi-Agent
3.	Orchestration Patterns & Topologies
4.	Communication, Coordination & Interop Protocols (MCP, A2A, handoffs)
5.	Frameworks & SDKs Landscape
6.	Context Engineering & Memory for Agents
7.	Reliability, Evaluation & Observability
8.	Cost, Performance & Economics of Multi-Agent Systems
9.	Applied: Agent Orchestration in Print-Flow-360

Agent Orchestration & Multi-Agent Systems — Suite Index

Source: [readme/research/agent-orchestration/AGENT_ORCHESTRATION_00_INDEX_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a reading guide for an 8-part research series about how to use AI assistants (called “agents”) to do complex work automatically. It explains when to use one AI versus many AIs working together, and why using more AIs is not always better. It matters because it helps you make smarter, cheaper decisions before building anything with AI.

The main parts explained simply:

- **When to use AI at all** — Not every task needs a smart AI. Sometimes a simple, fixed set of steps works better and costs far less. This series shows you a four-rung ladder: plain question → fixed steps → one AI assistant → many AI assistants working together. Always start at the bottom rung.
- **The 15x cost warning** — Using multiple AI assistants at once costs roughly 15 times more than asking one AI a single question. You only get that money's worth if the job is genuinely valuable AND the parts can be done at the same time without interfering with each other. Most tasks do not meet this bar.
- **Patterns — ways to arrange AI assistants** — There are 11 named ways to set up AI assistants (for example: one AI hands work to another, or several AIs vote on an answer). Each pattern fits certain jobs and fails at others. The series explains when to use each one and what can go wrong.
- **How AI assistants talk to each other and to tools** — Agents need to share information and use outside tools (like searching the web or reading a file). This part covers the rules and formats that make that work reliably.
- **Memory and attention limits** — An AI can only “hold” a certain amount of information in its head at one time. This part explains how to keep that space clean and useful so the AI does not get confused on long tasks.
- **Reliability and catching mistakes** — The longer an AI works on a task, the more small errors can pile up into a big problem. This part covers how to catch mistakes early, test AI systems, and make sure a long job can recover if something goes wrong midway.
- **Controlling the cost** — Token usage (the unit you pay for with AI APIs) can spiral fast. Techniques like caching repeated instructions (90% discount) and batching requests (50% off) can change the economics significantly.
- **How this applies to Print-Flow-360 (PF360)** — This part maps all the abstract ideas onto the actual AI code already in this platform. It also lists four realistic improvements that could be built, and flags what must be fixed first before any of them are possible.

What to do with this:

Read part 01 (Foundations) first — it gives you the decision framework you need before anything else makes sense. If you want to know what is possible in this codebase right now, jump to part 08 (Applied) directly after.

Entry point and reading map for an 8-part deep dive on building, running, and paying for multi-agent LLM systems.

Executive summary

Agent orchestration is the discipline of coordinating one or more LLM-driven agents — each running a `perceive → plan → act → observe` loop over tools — to accomplish a task that a single model call cannot reliably finish on its own. “Multi-agent” specifically means splitting that work across several agents (an orchestrator plus workers, a supervisor tree, a debate, a handoff swarm) rather than one agent grinding through every step.

The core insight that runs through every part of this suite:

Add agentic complexity only when it pays for itself. Climb the ladder one rung at a time — a single well-prompted LLM call beats a workflow, a workflow beats an agent, and a single agent beats a multi-agent system — *until the task’s structure forces you up*. Most production “AI features” should be a prompt or a fixed workflow, not an autonomous agent, and almost never a swarm.

The single biggest tradeoff you are buying when you go multi-agent:

Roughly a 15× token cost versus a single chat turn (Anthropic measured ~4× for a single agent and ~15× for their multi-agent research system). You only recoup that when the work is **high-value and parallelizable** — breadth-first research, fan-out over many independent sources, sweeps with disjoint ownership. For interdependent reasoning under a fixed budget (most coding, most tightly-coupled planning), a single agent wins on both cost *and* quality, because splitting it just multiplies coordination failures.

Everything else in the suite is the detail behind those two sentences: which pattern fits which task (02), how agents talk to each other and to tools (03), what framework to reach for (04), how to keep a finite context window useful across a long run (05), how to make a non-deterministic long-horizon system reliable and observable (06), how to control the cost (07), and how all of it lands on Print-Flow-360’s actual code (08).

Read in this order

#	File	What it covers
01	Foundations: Agents, Agentic Loops & When to go Multi-Agent	Vocabulary + decision framework: the three-rung ladder (call → workflow → agent), the augmented LLM, the ReAct loop, the four “agentic” properties, Anthropic’s five workflow patterns, and the single-vs-multi gate with measured costs. Start here.
02	Orchestration Patterns & Topologies	Eleven patterns along the determinism → autonomy axis — chaining, routing, parallelization, orchestrator-worker, evaluator-optimizer, supervisor trees, blackboard, handoff swarms, debate, map-reduce, reflection loops — each with an ASCII topology, when-to-use/when-not, and failure modes.
03	Communication, Coordination & Interop Protocols	How agents coordinate: message-passing vs shared-memory blackboards vs LangGraph state, handoff semantics, task decomposition + result aggregation, and the open standards MCP (agent ↔ tools) and A2A (agent ↔ agent), plus structured-output messaging pitfalls.
04	Frameworks & SDKs Landscape	Nine frameworks (LangGraph, CrewAI, AutoGen/AG2, OpenAI Agents SDK, Claude Agent SDK, Google ADK, Semantic Kernel, LlamaIndex) on an explicit → emergent control spectrum, with “use it when / avoid when” and a selection algorithm. Opens with “do you even need a framework?”
05	Context Engineering & Memory for Agents	Managing the finite attention budget: reduce / offload / retrieve / isolate, context rot & lost-in-the-middle, memory tiers (short/long, semantic/episodic/procedural), RAG vs agentic retrieval, sub-agent isolation, and prompt-caching mechanics.
06	Reliability, Evaluation & Observability	Why compounding per-step error breaks long runs, the MAST 14-failure-mode taxonomy, layered guardrails, outcome vs trajectory evals + LLM-as-judge, OpenTelemetry GenAI tracing, non-determinism testing, and resumable/durable execution.
07	Cost, Performance & Economics	The ~15× multiplier in detail, token spend as ~80% of performance variance, when multi-agent pays off, and the control levers: model tiering, prompt caching (90% input discount), Batch API (50% off), rate-limit management, and runtime budget guardrails.
08	Applied: Agent Orchestration in Print-Flow-360	How the patterns map onto PF360’s real AI code (<code>AiTaskRunner</code> + 7 consumers), the four hard constraints (no tool-use, no caching, no live-catalog injection, <code>QUEUE=sync</code>), and four realistic-but-unbuilt orchestration opportunities. Research-only.

Cheat sheet (one page)

Decision flow — single call → workflow → single agent → multi-agent

```
Start: what is the task?
|
|— Can one well-crafted prompt do it reliably?           → PLAIN LLM CALL.  Stop.
|   (classify, extract, rewrite, summarize, draft)
|
|— Are the steps known and fixed in advance?             → WORKFLOW.  Stop.
|   (chain / route / parallelize / evaluator-optimizer
|    on a predetermined graph – deterministic control flow)
|
|— Does the path depend on intermediate results,         → SINGLE AGENT (one ReAct loop, tools).
|   needing dynamic tool use & self-correction?         Stop here unless a gate below trips.
|
|— Do ALL of these hold?                                 → MULTI-AGENT (orchestrator-worker etc.)
|   • the work is HIGH-VALUE (worth ~15× tokens)
|   • it is genuinely PARALLELIZABLE (independent subtasks)
|   • a single context window can't hold it (overload)
|   • subtasks have clean, disjoint ownership
|   If any is NO → stay with a single agent.
```

Two gates justify the jump to multi-agent, and you need **both**:

- **Economic gate** — the output is valuable enough to absorb a ~15× token bill.
- **Overload / parallelism gate** — one agent's context or serial latency is the bottleneck, and subtasks are independent enough to run side by side without constant coordination.

Pattern picker (condensed — full version in part 02)

Pattern	Parallel?	Control	Relative cost	Best for
Prompt chaining	No	Deterministic	Low	Fixed multi-step transforms (outline → draft → polish)
Routing	No	Deterministic	Low	Classify input, dispatch to a specialized handler
Parallelization (sectioning / voting)	Yes	Deterministic	Medium	Independent subtasks, or N votes for reliability
Orchestrator-worker	Yes	Dynamic	High (~15×)	Breadth-first research, dynamic fan-out over sources
Evaluator-optimizer	No	Loop	Medium	Output has clear quality criteria; iterate to a bar
Supervisor tree	Yes	Hierarchical	High	Large tasks needing layered delegation
Blackboard / shared state	Yes	Emergent	High	Many agents collaborating on one evolving artifact
Handoff / swarm	No	Decentralized	Medium-High	Specialist takes full control of a phase, then hands off
Debate / ensemble	Yes	Voting	High	High-stakes judgment where diversity beats one opinion
Map-reduce	Yes	Deterministic	Scales w/ corpus	Summarize / extract over a large document set
Reflection loop (ReAct / Reflexion)	No	Loop	Medium	Single agent self-correcting until done

Rule of thumb: **prefer the lowest row that fits, and prefer deterministic control over emergent**. Reach for the high-cost rows only when both gates above are satisfied.

Top 8 do's

1. **Start at the simplest rung** that could plausibly work; add complexity only when you've watched the simpler version fail for a structural reason.
2. **Make control flow as deterministic as the task allows** — a workflow you can read beats an agent you have to debug.
3. **Tier your models** — a strong orchestrator with cheap workers captures most of the quality at a fraction of the cost (part 07).
4. **Engineer the context, not just the prompt** — reduce / offload / retrieve / isolate to keep the window's signal high (part 05).

5. **Give subagents disjoint ownership** and clear contracts so their outputs merge cleanly without conflict.
6. **Instrument from day one** — OpenTelemetry GenAI traces, trajectory + outcome evals, small high-signal eval sets (part 06).
7. **Design for resume-not-restart** — checkpoint state so a long run can recover from a failed step (part 06).
8. **Exploit caching and batching** — a stable cached prefix (90% input discount) and the Batch API (50% off) change the economics materially (part 07).

Top 8 don'ts

1. **Don't reach for multi-agent by default** — it's the most expensive and failure-prone option, not the impressive one.
2. **Don't split interdependent reasoning** across agents — coordination overhead and context loss will cost you quality (coding is the classic trap).
3. **Don't let context rot** — never assume more tokens = better; mid-context info degrades ("lost in the middle").
4. **Don't share one mutable context across agents** carelessly — context pollution and error blast radius compound silently.
5. **Don't ship without guardrails** — input/action/output/permission checks and human-in-the-loop on irreversible actions.
6. **Don't treat a multi-agent system as deterministic** in tests — design for non-determinism and assert on outcomes/invariants.
7. **Don't pick a heavyweight framework before you need one** — Anthropic's own advice: start without one (parts 01, 04).
8. **Don't ignore the bill** — token spend is ~80% of performance variance; a runaway loop is a runaway invoice (part 07).

How this maps to Print-Flow-360

If you're here to ship something in this codebase rather than to study the field, **read part 08** after part 01. It grounds every abstract pattern in PF360's actual AI surface: the single stateless `AITaskRunner` wrapping per-tenant Claude Sonnet / OpenAI providers behind seven feature consumers, and the four hard constraints that shape what's even possible today — **no native tool-use, no prompt caching, no live-catalog injection, and `QUEUE=sync` forcing inline runs**. It then sketches four realistic-but-unbuilt orchestration opportunities (orchestrator-worker product builder, content/SEO chaining, support triage, evaluator-optimizer design loop), each tied to a concrete code seam and gated on the same shared enablers (Redis async, tool-use, prompt caching).

Part 08 is **explicitly research-only** — nothing in this suite has been built into PF360. It also flags two correctness traps: image generation is stubbed, and the AI matcher docs describe *intent*, not shipped

code.

Sources

This index summarizes the eight parts; each part carries its own full source list. The load-bearing primary sources across the suite:

- Building effective agents — Anthropic — <https://www.anthropic.com/research/building-effective-agents>
- How we built our multi-agent research system — Anthropic — <https://www.anthropic.com/engineering/built-multi-agent-research-system>
- Effective context engineering for AI agents — Anthropic — <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Why Do Multi-Agent LLM Systems Fail? (MAST taxonomy) — Cemri et al. — <https://arxiv.org/abs/2503.13657>
- ReAct: Synergizing Reasoning and Acting in Language Models — Yao et al. — <https://arxiv.org/abs/2210.03629>
- Lost in the Middle: How Language Models Use Long Contexts — Liu et al. — <https://arxiv.org/abs/2307.03172>
- Context Rot — Chroma Research — <https://research.trychroma.com/context-rot>
- Model Context Protocol (MCP) — <https://modelcontextprotocol.io/>
- Agent2Agent (A2A) Protocol — <https://a2aproject.github.io/A2A/>
- OpenTelemetry GenAI semantic conventions — <https://opentelemetry.io/docs/specs/semconv/gen-ai/>
- Prompt caching — Anthropic docs — <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>
- LangGraph documentation — <https://langchain-ai.github.io/langgraph/>
- OpenAI Agents SDK — <https://openai.github.io/openai-agents-python/>

Foundations: Agents, Agentic Loops & When to go Multi-Agent

Source: [readme/research/agent-orchestration/AGENT_ORCHESTRATION_01_FOUNDATIONS_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document explains what an “AI agent” actually is — and when it makes sense to build one. The word “agent” gets used loosely, so this doc gives you a clear way to think about it before you start building. The big idea: always use the simplest solution that works. Only add more complexity when the benefit is obvious.

The main parts explained simply:

- **Three levels of AI systems** — There is a simple scale: (1) a single AI question-and-answer, (2) a “workflow” where the steps are fixed in code and the AI just fills in the blanks, and (3) a true “agent” where the AI decides its own next steps as it goes. Most tasks only need level 1 or 2.
- **The agentic loop** — What makes something an “agent” is that it works in a repeating cycle: look at the task → decide what to do → take an action (like searching the web or running a calculation) → read the result → repeat until done. This is different from a one-shot question you ask and get one answer.
- **ReAct: the simplest recipe for an agent** — A research paper called ReAct showed that if you let an AI write down its thinking (“thought”), take an action, then read the result (“observation”), and repeat, it makes far fewer mistakes than just asking it once. It outperformed older methods by 10–34%.
- **Five workflow patterns** — Before building a full agent, consider simpler fixed patterns: chaining steps in order, routing different inputs to different paths, running tasks in parallel, using one AI to coordinate others, or having one AI check another’s work. Most “I need an agent” problems are actually one of these, which are cheaper and more predictable.
- **Single agent vs multiple agents** — Using multiple AI agents working together can improve results dramatically (one study: 90% improvement on research tasks), but it costs roughly 15 times more to run than a simple AI chat. Coordination between agents also breaks down in many ways — about a third of multi-agent failures are just agents misunderstanding each other.
- **The decision ladder** — A simple checklist to decide how much AI you actually need: Can one good AI call solve it? → stop there. Are the steps fixed? → use a workflow. Does the AI need to figure out its own path? → use a single agent. Is that agent overloaded with too many tools, and is the task high-value enough to justify the cost? → only then use multiple agents.
- **Tool design matters** — The tools an agent can use (search, calculate, send emails, etc.) need to be designed carefully. One team found that improving tool descriptions alone cut task completion time by 40%.

What to do with this:

Start with the simplest thing that solves your problem — a good prompt or a fixed script. Only move to agents or multi-agent systems when the task genuinely cannot be handled with simpler steps, and only when the value of the task justifies the significantly higher cost.

The vocabulary, the loop mechanic, and the decision ladder you climb before ever spawning a second agent.

Why this doc exists

“Agent” and “multi-agent” are thrown around loosely. Before you build anything, you need a precise mental model of (1) what separates a plain LLM call from a workflow from an agent, (2) the loop that makes something “agentic,” and (3) the hard economic and reliability gates that decide whether you need *one* agent, *many* agents, or *no* agent at all.

The unifying thesis across every serious source (Anthropic, OpenAI, Google) is the same: **use the simplest thing that works; add agentic complexity only when it clearly pays off**. This doc gives you the framework to make that call deliberately rather than by hype.

1. The three-rung ladder: plain call → workflow → agent

There is a spectrum of increasing autonomy. The sharpest industry line (Anthropic) is drawn between **workflows** and **agents** — both of which sit under the umbrella term **agentic systems**.

Term	Definition	Who controls the flow	Predictability
Plain LLM call	A single model invocation (optionally augmented with retrieval/tools/memory), one turn, no loop. Apps that “integrate LLMs but don’t use them to control workflow execution — think simple chatbots, single-turn LLMs, or sentiment classifiers — are not agents.” (OpenAI)	Developer / code	Highest
Workflow	“Systems where LLMs and tools are orchestrated through predefined code paths .” (Anthropic) The developer fixes the sequence; the LLM fills in steps.	Developer-defined code path	High
Agent	“Systems where LLMs dynamically direct their own processes and tool usage , maintaining control over how they accomplish tasks.” (Anthropic)	The LLM (model-driven)	Lowest / dynamic

The three canonical definitions, side by side:

- **Anthropic:** “Workflows are systems where LLMs and tools are orchestrated through predefined code paths. Agents, on the other hand, are systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks.”
- **OpenAI:** “Agents are systems that **independently** accomplish tasks on your behalf.” Two defining traits: (1) it “leverages an LLM to manage workflow execution and make decisions... recognizes when a workflow is complete and can proactively correct its actions”; (2) it “has access to various tools... and dynamically selects the appropriate tools depending on the workflow’s current state.”

- **Google (Agents whitepaper):** “A Generative AI Agent... is an application that attempts to achieve a goal by **observing the world and acting upon it using tools** that it has at its disposal.” The differentiator from a bare model is an **orchestration layer** enabling “iterative reasoning and multi-step problem-solving,” unlike models that “operate on single-turn predictions.”

Takeaway: “Agentic” is a spectrum, not a binary. A plain call *answers*; a workflow *follows a script the developer wrote*; an agent *decides its own path at runtime*.

2. The building block underneath everything: the augmented LLM

Every agentic system — workflow or agent — is composed from one foundational unit: the **augmented LLM**, an LLM enhanced with **retrieval, tools, and memory** (Anthropic). Modern models “actively generate search queries, select appropriate tools, and determine what information to retain.”

The same idea appears as a three-part cognitive architecture across vendors:

Anthropic	Google (“cognitive architecture”)	OpenAI (“core components”)	Role
The model + tools + memory	Model — the “brain,” central decision-maker	Model — reasoning/decision-making	Decide
(tools)	Tools — “hands and eyes” (Extensions, Functions, Data Stores)	Tools — external functions/APIs to take action	Act
(memory/loop)	Orchestration Layer — the “nervous system”: memory, state, reasoning, planning; drives the loop via ReAct / Chain-of-Thought / Tree-of-Thought	Instructions — guidelines & guardrails defining behavior	Govern / loop

If a single well-engineered augmented-LLM call (good prompt + retrieval + in-context examples) solves your task, **stop there**. Everything below is added complexity you must justify.

3. The agentic loop (perceive → plan → act → observe)

The defining mechanic of an agent is a **loop, not a pipeline**.



Anthropic’s description of the loop: agents “begin their work with either a command from, or interactive discussion with, the human user. Once the task is clear, agents plan and operate independently, potentially returning to the human for further information or judgement.” Critically, “at each step... the agent gains **‘ground truth’ from the environment** (such as tool call results or code execution) to assess its progress.” Agents can pause for human checkpoints or on hitting blockers, and typically “terminate upon completion, but it’s also common to include stopping conditions (such as a maximum number of iterations).”

OpenAI’s framing of the same loop: a single agent “runs in a loop” until an exit condition is met. Exit conditions “include tool calls, a certain structured output, errors, or reaching a maximum number of turns.”

ReAct — the canonical instantiation of the loop

- **Paper:** *ReAct: Synergizing Reasoning and Acting in Language Models* — Yao et al., submitted Oct 6 2022 (arXiv:2210.03629), ICLR 2023.
- **Core idea:** interleave **reasoning traces** (“thoughts”) with **task-specific actions**. The loop is concretely **Thought → Action → Observation**, repeated. “Reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with external sources to gather additional information.”
- **Why it matters:** ReAct “overcomes issues of hallucination and error propagation prevalent in chain-of-thought reasoning by interacting with a simple Wikipedia API.” Reasoning grounds action; observation from the environment corrects reasoning. Trajectories are “more interpretable” and improve “human interpretability and trustworthiness.”
- **Results:** on the interactive benchmarks ALFWorld and WebShop, ReAct beat imitation/RL baselines by an absolute success rate of **34%** and **10%** respectively; also evaluated on HotpotQA and FEVER.

Mental model: the generic loop (perceive → plan → act → observe) is the *spine* of every agent. ReAct is the simplest concrete recipe for it. The closed loop — observing the *result* of each action and adapting — is what separates an agent from an open-loop pipeline.

4. What actually makes something “agentic” — four properties

Synthesizing across sources, four properties distinguish agentic systems from plain LLM use. The more present, the more “agentic”:

1. **Autonomy / dynamic control flow** — the LLM, not hard-coded logic, decides what to do next and how many steps to take. The number of steps and the solution path *cannot be predetermined*. (Anthropic: “dynamically direct their own processes”; OpenAI: “recognizes when a workflow is complete.”)
2. **Tool use** — the system can *act on the world*, not just emit text (Google’s “hands and eyes”).
3. **Environment feedback grounding** — the agent observes the *results* of its actions (tool outputs, code execution, errors) and adapts. Anthropic calls this gaining “ground truth from the environment.” This is the open-loop vs closed-loop difference.
4. **Iteration toward a goal** — operates in a loop until a goal/stop-condition is met, rather than one-shot.

System	Dynamic control flow	Tool use	Env. feedback	Iteration
Plain classifier	—	—	—	—
RAG call	—	✓	—	—
Workflow	— (fixed in code)	✓	✓	✓
True agent	✓	✓	✓	✓

The single property that promotes a workflow to an agent is **dynamic control flow**: in a workflow the steps are fixed in code; in an agent the model chooses them at runtime.

5. The five canonical workflow patterns

Before reaching for a fully autonomous agent, Anthropic catalogs five **workflow** patterns that are more predictable and cheaper. (LangGraph implements exactly these.) Most “I need an agent” problems are actually one of these.

Pattern	What it does	When to use
Prompt chaining	Decompose into a fixed sequence of LLM calls; each processes the prior output; programmatic “gate” checks between steps.	Task cleanly decomposes into fixed subtasks; trade latency for accuracy.
Routing	Classify the input, then direct it to a specialized follow-up prompt/path.	Distinct input categories better handled separately; allows separate optimization per route.
Parallelization	Run subtasks simultaneously. Two forms: Sectioning (independent subtasks) and Voting (same task run multiple times for diverse/consensus output).	Subtasks are independent, or you want multiple attempts / guardrail votes for confidence.
Orchestrator–workers	A central LLM dynamically breaks down the task, delegates to worker LLMs, then synthesizes results.	Complex tasks where the subtasks can’t be predicted up front .
Evaluator–optimizer	One LLM generates; a second evaluates and gives feedback; loop until criteria met.	Clear evaluation criteria exist and iterative refinement measurably helps.

Key nuance — the bridge from workflow to agent: orchestrator–workers $\neq$ parallelization. In parallelization the subtasks are **fixed in code**; in orchestrator–workers the lead LLM **decides the subtasks at runtime**. That runtime decision is exactly the “dynamic control flow” property — which is why orchestrator–workers is the conceptual gateway to multi-agent systems.

6. Single-agent vs multi-agent

What multi-agent actually buys you

- **Breadth and parallel exploration.** Anthropic’s multi-agent **Research** system (orchestrator–worker: a Lead Researcher spawns **3–5 subagents** in parallel, each using **3+ tools in parallel**) scored a **90.2% improvement over single-agent Claude Opus 4** on their internal research eval. It “excels at breadth-first queries that involve pursuing multiple independent directions simultaneously.”
- **Context-window scaling.** Subagents have their own context windows, so the system can process information that “exceeds single context windows.”
- **Speed via parallelism.** Parallel tool calling “cut research time by up to 90% for complex queries.”
- **Separation of concerns.** More agents “can provide intuitive separation of concepts” (OpenAI).

The real costs of multi-agent

Cost	Detail (Anthropic, measured)
Token multiplier	Agents use ~4× more tokens than chat. Multi-agent systems use ~15× more tokens than chat . In their BrowseComp analysis, token usage alone explained ~80% of the variance in performance (model choice and tool-call count were secondary).
Economic gate	“Multi-agent systems require tasks where the value of the task is high enough to pay for the increased performance .” Below that threshold, don’t.
Coordination overhead	“LLM agents are not yet great at coordinating and delegating to other agents in real time.” Lead agents currently run subagents synchronously (waiting for each batch), creating bottlenecks.
Error compounding	Agents “run for long periods... maintaining state across many tool calls... minor system failures can be catastrophic.” Errors propagate across long trajectories.
Non-determinism	“Agents make dynamic decisions and are non-deterministic between runs, even with identical prompts. This makes debugging harder.” Deployment needs care (e.g. “rainbow deployments” to avoid disrupting in-flight agents).

When multi-agent does NOT fit (Anthropic, explicit)

“Some domains that require all agents to **share the same context** or involve **many dependencies between agents** are not a good fit for multi-agent systems today.”

Specifically called out: **most coding tasks** “involve fewer truly parallelizable tasks than research.” Good fit = high-value tasks with heavy parallelization, info exceeding one context window, and many complex tools.

Independent evidence that multi-agent often underperforms

The Berkeley **MAST** study (*Why Do Multi-Agent LLM Systems Fail?*, Cemri, Pan, Yang et al., arXiv:2503.13657, NeurIPS 2025) analyzed **7 popular MAS frameworks** across **1,600+ annotated traces** (inter-annotator $\kappa = 0.88$) and found “performance gains on popular benchmarks are often minimal.” It identifies **14 failure modes in 3 categories**:

1. **System design / specification issues** — e.g. step repetition (15.7%), unaware of termination conditions (12.4%), disobeying task spec (11.8%).
2. **Inter-agent misalignment** — communication/coordination breakdowns, **~32% of failures**.
3. **Task verification failures** — inadequate output validation, error propagation.

The lesson: adding agents adds *new* failure surfaces (coordination, role confusion, premature/late termination) **on top of** all the single-agent ones. More agents is not free quality.

7. The decision ladder — do you even need an agent (or more than one)?

The unifying stance: **use the simplest thing that works; add agentic complexity only when it clearly pays off**. At every rung, the burden of proof is on the *added* complexity.

```
START
|
| — Single well-engineered LLM call (prompt + retrieval + examples) — enough? — YES → ship it.
|                                                                 (Anthropic: "this
might mean not building agentic systems at all.")
| NO
|
| ▼
| — Are the steps fixed & knowable? — YES → WORKFLOW (chaining / routing / parallelization).
|                                     Predictable, cheaper, easier to debug.
| NO (path is genuinely unpredictable; needs runtime decisions)
|
| ▼
| — SINGLE AGENT with tools in a loop. ← maximize this first.
|
| — Is the single agent overloaded? (>~15 distinct tools, OR overlapping/similar tools,
|   OR sprawling if-then-else branching) — NO → stay single-agent.
| YES
|
| ▼
| — AND is the task high-value, parallelizable, and exceeding one context window?
|   — NO → fix the single agent (better tools/prompts), don't split.
|   — YES → MULTI-AGENT (manager or handoff pattern).
```

When agents earn their keep (OpenAI’s three signals)

- 1. **Complex decision-making** — nuanced judgment, exceptions, context-sensitive calls (e.g. refund approval).
- 2. **Difficult-to-maintain rules** — rule systems grown “unwieldy,” costly/error-prone to update.
- 3. **Heavy reliance on unstructured data** — interpreting natural language, extracting from documents, conversational interaction.

If a problem hits *none* of these, a deterministic/rules-based solution is probably better than any agent.

Workflow vs agent (Anthropic)

- Use **workflows** for “predictability and consistency for well-defined tasks.”
- Use **agents** when you need “flexibility and model-driven decision-making at scale” — open-ended problems where you can’t predict the number of steps or hard-code the path.

Concrete triggers to split into multiple agents (OpenAI)

- **Tool overload / overlap:** the problem “isn’t solely the number of tools, but their similarity or overlap. Some... manage more than **15 well-defined, distinct tools** while others struggle with fewer than 10 overlapping tools.” When a single agent can’t reliably pick the right tool → split.
- **Complex conditional logic:** when prompts fill with many if-then-else branches and templates “get difficult to scale,” divide each logical segment into its own agent.

The two multi-agent orchestration patterns (OpenAI), once you do split

Pattern	How it works	Best when
Manager	A central “manager” agent coordinates specialized agents via tool calls (agents-as-tools); one agent keeps central control and user contact. Edges = tool calls.	You want one controller synthesizing results and owning the conversation.
Decentralized / handoff	Agents “hand off” execution to one another; edges = handoffs that transfer control.	No single agent needs central control; each can take over interacting with the user.

The economic gate (Anthropic), restated

Even when multi-agent improves quality, only deploy it where the **task value justifies the ~15× token cost**. High-value research/analysis: yes. Routine, low-margin, or tightly-coupled tasks: no.

8. Supporting detail: tool engineering (the agent–computer interface)

Anthropic stresses the **agent–computer interface (ACI)** deserves as much investment as prompt engineering: “we’ve seen developers spend more effort on prompts than tools.”

- Format tools to match how they appear “naturally” in text; avoid overhead like counting lines or escaping.
- Give the model enough tokens to “think” before it commits to an action.
- Write thorough tool docs with examples and edge cases; apply **poka-yoke** — design the tool so misuse is hard.
- Test extensively. In Anthropic’s multi-agent system, a **tool-testing agent that rewrote flawed tool descriptions produced a 40% decrease in task completion time** for later agents.

Three core principles for agent success (Anthropic): **Simplicity**, **Transparency** (show the planning steps explicitly), and a well-crafted **ACI**.

9. One-paragraph summary to carry into parts 02–08

Start with a single, well-engineered **LLM call** + good prompt/retrieval. If the task has fixed, knowable steps → build a **workflow** (chaining / routing / parallelization). If the path is genuinely unpredictable and needs runtime decisions → build a **single agent** with tools in a loop (the perceive → plan → act → observe loop, instantiated as ReAct). Only when a single agent is overloaded (>~15 distinct tools, tool overlap, sprawling branching) **and** the task is high-value, parallelizable, and exceeds one context window → go **multi-agent** (manager or handoff). The agentic property that promotes each rung is **dynamic control flow**; the gate that stops you climbing too far is the **~15× token cost** and the **new coordination failure modes** that MAST documents. At every rung, complexity must clearly pay off.

Sources

- Building Effective Agents — Anthropic Engineering — <https://www.anthropic.com/engineering/building-effective-agents>
- How we built our multi-agent research system — Anthropic Engineering — <https://www.anthropic.com/engineering/multi-agent-research-system>
- Anthropic: How we built our multi-agent research system (summary w/ token-multiplier & 90.2% quotes) — Simon Willison — <https://simonwillison.net/2025/Jun/14/multi-agent-research-system/>
- A practical guide to building agents (PDF) — OpenAI — <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>
- A practical guide to building agents (Markdown mirror, used for exact quotes) — <https://gist.github.com/testy-cool/86cafd426ba22e3e8c1d6d2c853506c4>
- A practical guide to building agents (landing) — OpenAI — <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>
- ReAct: Synergizing Reasoning and Acting in Language Models — Yao et al., arXiv:2210.03629 — <https://arxiv.org/abs/2210.03629>
- ReAct paper PDF — <https://arxiv.org/pdf/2210.03629>

- ReAct: Synergizing Reasoning and Acting in Language Models — Google Research blog — <https://research.google/blog/react-synergizing-reasoning-and-acting-in-language-models/>
- Workflows and Agents (tutorial, implements Anthropic's patterns) — LangGraph docs — <https://langchain-ai.github.io/langgraph/tutorials/workflows/>
- Google "Agents" Whitepaper (definition + Model/Tools/Orchestration cognitive architecture; summary) — <https://medium.com/@penkow/summary-of-googles-ai-white-paper-agents-d5670ae495c9>
- Why Do Multi-Agent LLM Systems Fail? (MAST taxonomy) — Cemri, Pan, Yang et al., arXiv:2503.13657 — <https://arxiv.org/abs/2503.13657>
- MAST taxonomy repo — <https://github.com/multi-agent-systems-failure-taxonomy/MAST>

Orchestration Patterns & Topologies

Source: [readme/research/agent-orchestration/AGENT_ORCHESTRATION_02_PATTERNS_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a catalogue of the different ways you can wire multiple AI “agents” together to get work done. Think of it like choosing the right factory floor layout — depending on the job, you arrange workers differently. Knowing these patterns helps you pick the right one instead of defaulting to the most complicated (and expensive) option.

The main parts explained simply:

- **Workflows vs. Agents** — A workflow follows a fixed script you write in advance; an agent figures out its own next steps on the fly. Fixed scripts are cheaper and more predictable. Agents are more flexible but cost more and make more mistakes. The advice: start simple, only add agent-style thinking when the fixed script clearly isn't good enough.
- **Prompt chaining (step-by-step pipeline)** — Pass the output of one AI call straight into the next, like an assembly line. Good for tasks with clear, fixed steps (write → translate → check). Fragile if an early step goes wrong, because the mistake flows through every step after it.
- **Routing / dispatch** — A first AI reads the incoming request and sends it to the right specialist. Like a receptionist who decides which department handles your call. Breaks down when the categories are fuzzy and the receptionist keeps sending people to the wrong desk.
- **Parallelization** — Run several AI calls at the same time and combine the results. Two flavours: split a big task into independent chunks (faster), or run the same task multiple times and take the majority answer (more accurate). Only works when the pieces genuinely don't depend on each other.
- **Orchestrator-worker (lead agent + helpers)** — A lead AI plans the job, splits it into subtasks on the fly, hands each to a helper AI, then combines everything. Anthropic's internal research tool uses this and runs up to 90% faster than doing it in sequence. Downside: it can wildly over-spawn helpers, helpers duplicate each other's work, and it uses roughly 15× more tokens (and cost) than a simple chat.
- **Evaluator-optimizer (writer + critic loop)** — One AI writes a draft; a separate AI critiques it; the writer revises. Loops until the critic is satisfied. Good when you can clearly define what “good enough” looks like. Risks: the loop never ends without a hard stop, and the two AIs can settle on a mediocre answer they both agree on.
- **Hierarchical / supervisor trees** — A top manager routes tasks to team supervisors, who each manage a group of specialist workers. Easy to audit and trace, but every extra level adds delay, and the top manager is a single point of failure.
- **Blackboard / shared-state** — All agents read and write to a shared noticeboard. No one talks directly to anyone else — agents just post updates and pick up whatever they can help with. Efficient for discovery-style tasks, but the scheduling logic (who writes next) is the hard part to get right.
- **Network / handoff (swarm)** — Agents hand control directly to each other, no manager in the middle. Faster and simpler than a supervisor tree for conversational tasks where the specialist should just answer the user directly. Hard to trace when something goes wrong because there is no central record of decisions.
- **Debate & ensemble voting** — Multiple AIs each give an answer, then critique each other's reasoning over several rounds until they agree. Reduces hallucination on hard factual or maths

questions. Expensive. Research also shows a single AI with the same token budget can sometimes beat a whole debate panel.

- **Map-reduce over large documents** — Break a huge document into chunks, summarize each chunk independently, then combine the summaries. The only practical way to handle content that is too long to fit in one AI call. Loses cross-document connections that only become visible when you read the whole thing.
- **Reflection / self-improvement loop (ReAct, Reflexion)** — A single AI acts, checks its own result, criticizes itself, and tries again — repeating until it meets a quality bar or hits a retry limit. Useful when there is a clear right/wrong signal (tests pass or fail). Risk: the AI may not see its own blind spots, and the loop can reinforce a wrong approach rather than escape it.

What to do with this:

Always start with the simplest pattern (a plain step-by-step chain or a single prompt). Only move to multi-agent, orchestrator-worker, or debate patterns when simpler approaches have clearly failed — those advanced patterns cost 10–15× more and introduce new ways to go wrong. When you do pick a more complex pattern, choose one where every decision is traceable (supervisor trees beat swarms for anything you need to audit).

A field guide to the standard ways LLM agents and workflows are wired together — what each pattern is, when to reach for it, and how it fails.

Research/reference doc · 2026-06-16 · part of the Agent Orchestration suite

0. Framing: workflows vs. agents (read this first)

The single most useful distinction in the literature, from Anthropic's *Building Effective Agents*:

- **Workflows** — “systems where LLMs and tools are orchestrated through **predefined code paths**.” Deterministic, reliable, built for well-defined tasks.
- **Agents** — “systems where LLMs **dynamically direct their own processes and tool usage**, maintaining control over how they accomplish tasks.” Flexible, model-driven, built for open-ended tasks.

Every pattern below sits somewhere on a single master axis running from **fully code-orchestrated** (deterministic, top of the list) to **fully LLM-orchestrated** (autonomous, bottom). OpenAI frames the identical axis as **orchestrate via code** (deterministic control flow; you control speed/cost/quality) vs. **orchestrate via LLM** (the agent plans its own next steps; best for open-ended work but needs heavy prompt/eval investment).

Anthropic's overarching guidance — keep this in front of you the whole time:

- Start with the **simplest optimized prompt**; add complexity (chaining → workflows → agents) **only when simpler approaches demonstrably underperform**.

- Three principles for agents: **maintain simplicity**, **prioritize transparency** (surface the planning steps), **carefully craft the agent-computer interface (ACI)** through good tool docs and testing.
- Agents carry **higher cost and the potential for compounding errors** — a bad early step poisons everything downstream.

DETERMINISTIC / code-orchestrated	AUTONOMOUS / LLM-orchestrated
Prompt chaining → Routing → Parallelization → Orchestrator-worker → Swarm/Blackboard	
cost ↑ , determinism ↓ , flexibility ↑ as you move right	

1. Prompt chaining / sequential pipeline

What it is. Decompose a task into **fixed sequential steps**, where each LLM call processes the previous call's output. Optional **programmatic gates** between steps validate intermediate results before continuing.

```

Input → [LLM call 1] → (gate/check) → [LLM call 2] → (gate) → [LLM call 3] → Output
      | fail
      ▼
    exit / fix
  
```

When to use. The task cleanly decomposes into **fixed subtasks**, and you're happy to trade latency for higher per-step accuracy. E.g. generate marketing copy → translate it; write an outline → validate it → draft the full document.

When NOT to use. The task isn't actually decomposable into fixed steps, or the steps vary by input. A rigid pipeline is brittle for open-ended work.

Failure modes.

- Latency accumulates **linearly** — every step is a full round-trip.
- Errors **propagate downstream**; a bad step-1 output poisons all later steps unless a gate catches it.
- Over-rigid for tasks whose shape varies per input.

2. Routing / dispatch

What it is. Classify the input, then **direct it to a specialized follow-up prompt / model / task**. Gives you separation of concerns and per-category optimization.

```

Input → [Router / |
        Classify] → [Specialist A]
                  ↗ [Specialist B]
                  ↘ [Specialist C]
  
```

When to use. Inputs fall into **distinct categories** better handled separately, **and** classification is accurate. Two flavours:

- **By category** — e.g. customer-service query types route to specialized prompts.
- **By difficulty / cost** — easy → cheap model (Haiku), hard → strong model (Sonnet/Opus). A simple, effective cost lever.

When NOT to use. Category boundaries are fuzzy or overlapping, or classification can't be done reliably — misroutes will dominate.

Failure modes.

- **Misclassification** sends input down the wrong branch — the dominant failure.
- The router is a **bottleneck and single point of failure**.
- Fuzzy/overlapping categories degrade accuracy.

3. Parallelization (sectioning + voting)

What it is. Run LLMs **simultaneously** and aggregate the outputs **programmatically**. Two variants:

- **Sectioning** — split into **independent subtasks**, run in parallel, then combine.
- **Voting** — run the **same task multiple times** (varied prompts/temperatures), then aggregate (majority / threshold).

SECTIONING

```
graph LR
    Input --> Split
    subgraph Parallel
        direction TB
        S1["↳ [subtask 1] ↴"]
        S2["↳ [subtask 2] ↴"]
        S3["↳ [subtask 3] ↴"]
    end
    Split --> S1
    Split --> S2
    Split --> S3
    S1 --> Aggregate
    S2 --> Aggregate
    S3 --> Aggregate
    Aggregate --> Out
```

VOTING

```
graph LR
    Input --> FanOut
    subgraph Parallel
        direction TB
        R1["↳ [run A] ↴"]
        R2["↳ [run B] ↴"]
        R3["↳ [run C] ↴"]
    end
    FanOut --> R1
    FanOut --> R2
    FanOut --> R3
    R1 --> Vote
    R2 --> Vote
    R3 --> Vote
    Vote --> Out
```

When to use. Speed (independent chunks) or higher confidence (multiple perspectives). Sectioning examples: a guardrail content-screen running alongside the main response; multi-aspect evaluation. Voting examples: code-vulnerability review by several reviewers; content-appropriateness behind a vote threshold.

Distinction vs. orchestrator-worker: parallelization uses **pre-defined, fixed** parallel paths. Orchestrator-worker decides the subtasks **dynamically at runtime**.

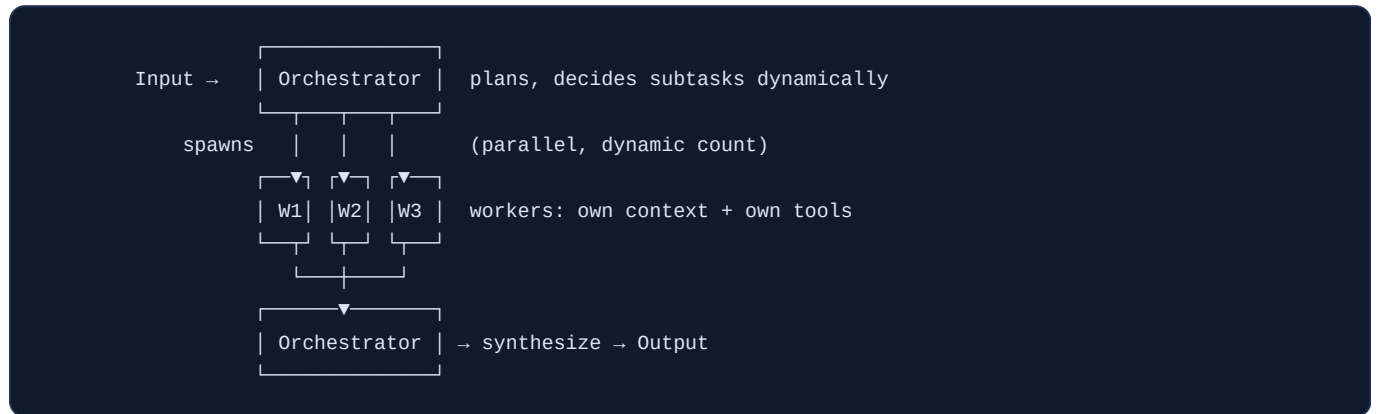
Failure modes.

- Aggregation is non-trivial — voting can **wash out a correct minority** answer.
- Cost multiplies by N (voting especially).
- Valid **only when subtasks are truly independent**; hidden dependencies cause inconsistent merges.

Academic analog: self-consistency decoding (aggregating multiple chain-of-thought samples) is the reasoning-level version of voting — it lowers individual error and raises confidence when paths converge.

4. Orchestrator-worker (lead agent spawns subagents)

What it is. A central lead LLM dynamically breaks the task into unpredictable subtasks, delegates each to a worker LLM, then **synthesizes** the results. The defining difference from parallelization: the subtasks are **not predetermined** — the orchestrator chooses them from the specific input.



When to use. Complex tasks where subtasks **can't be predetermined** — multi-file coding changes, multi-source gathering and analysis.

Failure modes (generic). Dynamic task count can explode; coordination overhead; the synthesis step can lose detail. See §4a for what this looks like in production.

4a. Anthropic's multi-agent research system — the flagship case study

This is the production instantiation of orchestrator-worker. The numbers here are worth memorizing.

Architecture.

- **Lead agent (orchestrator):** analyzes the query, develops a strategy, **saves its plan to memory**, spawns subagents, synthesizes their results, decides whether more research is needed, compiles the final answer.
- **Subagents (workers):** each gets its **own context window** and **own tools**; they act as “intelligent filters” — search independently, evaluate quality, identify gaps, return refined findings.
- **Models:** Claude Opus 4 as lead, Claude Sonnet 4 as subagents.

Two levels of parallelism.

1. Agent-level: lead “spins up 3–5 subagents in parallel rather than serially.”
2. Tool-level: each subagent fires **3+ parallel tool calls**. → cut research time by **up to 90%** on complex queries (minutes vs. hours).

Performance numbers.

- Multi-agent (Opus lead + Sonnet subagents) **beat single-agent Opus 4 by 90.2%** on the internal research eval.
- On BrowseComp, **3 factors explain 95% of performance variance; token usage alone explains 80%.**
- **Token economics:** agents use **~4×** the tokens of chat; **multi-agent systems use ~15×** the tokens of chat.
- “Upgrading to Claude Sonnet 4 is a larger performance gain than doubling the token budget on Sonnet 3.7.”

Coordination model. Currently **synchronous** — the lead waits for each batch of subagents to finish before continuing. Simpler to coordinate, but it creates bottlenecks: “the lead agent can’t steer subagents, subagents can’t coordinate, and the entire system can be blocked while waiting for a single subagent.”

Production failure modes observed.

- Lead **spawned 50 subagents for a simple query**; endless searching for nonexistent sources.
- **Duplicate work** — subagents exploring identical topics instead of dividing labor.
- Excessive status updates distracting agents.
- Agents are **stateful and error-prone** — “minor system failures can be catastrophic for agents”; errors compound across tool calls and phases.
- **Non-deterministic** runs make debugging hard → **full production tracing** required.
- **Tool-selection bias** — early agents “consistently chose SEO-optimized content farms over authoritative but less highly-ranked sources like academic PDFs or personal blogs.”

Seven prompt-engineering principles for orchestration.

1. **Simulate agent behavior** with real prompts/tools to find failure modes step-by-step.
2. **Detailed delegation** — give each subagent an objective, output format, tool guidance, and boundaries. Vague briefs (“research the semiconductor shortage”) cause misinterpretation and duplicate searches.
3. **Scale effort to complexity** — embed explicit rules: simple fact-find = 1 agent / 3–10 calls; comparisons = 2–4 subagents / 10–15 calls each; complex = 10+ subagents with divided responsibilities.
4. **Tool-design primacy** — distinct tool purposes; “bad tool descriptions can send agents down completely wrong paths.”
5. **Agentic self-improvement** — Claude can diagnose failing prompts; a tool-testing agent that rewrote tool descriptions produced a **40% decrease in task completion time.**
6. **Breadth-first then narrow** — start broad, evaluate, then focus.
7. **Extended thinking** as a controllable scratchpad; interleaved thinking in subagents to refine queries.

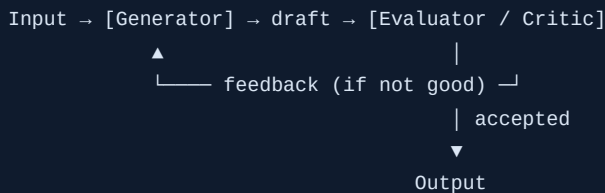
When multi-agent is the RIGHT fit: breadth-first queries pursuing multiple independent directions at once; high-value tasks needing heavy parallelization; information that exceeds a single context window; interfacing with many complex tools.

When it is the WRONG fit: tasks needing **shared context** across all agents; many **inter-agent dependencies**; **most coding tasks** (fewer parallelizable parts); real-time tight coordination; **low-**

value tasks where 15× token cost isn't justified. "Multi-agent systems require tasks where the value of the task is high enough to pay for the increased performance."

5. Evaluator-optimizer (generator + critic loop)

What it is. One LLM **generates**, a **second LLM evaluates and gives feedback**, in an **iterative loop**, until the evaluator is satisfied.



When to use. Clear **evaluation criteria exist** and iterative refinement demonstrably improves output. Two success signals: (a) responses measurably improve with human feedback; (b) the LLM can produce meaningful critiques. Examples: literary translation needing nuanced critique; multi-round complex search/analysis.

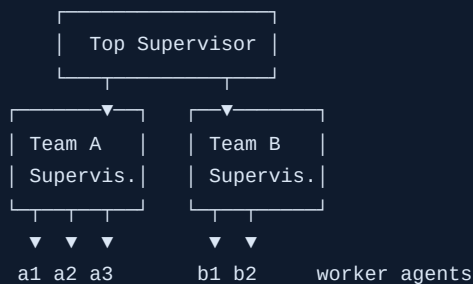
Distinction vs. §11 reflection: here a **separate** critic LLM judges the generator. In reflection, the **same agent** critiques itself.

Failure modes.

- Loops indefinitely without a stopping cap.
- Evaluator is only as good as its criteria — vague criteria → useless feedback.
- Two LLM calls per iteration → cost and latency multiply.
- Critic and generator can “collude” on a mediocre local optimum.

6. Hierarchical / supervisor trees

What it is. A **supervisor agent controls all communication and task delegation**, deciding which agent to invoke from context. **Hierarchical** = supervisors of supervisors: agents grouped into **teams**, each with its own supervisor, and a **top-level supervisor** routing between teams. Generalizes orchestrator-worker to multiple levels.



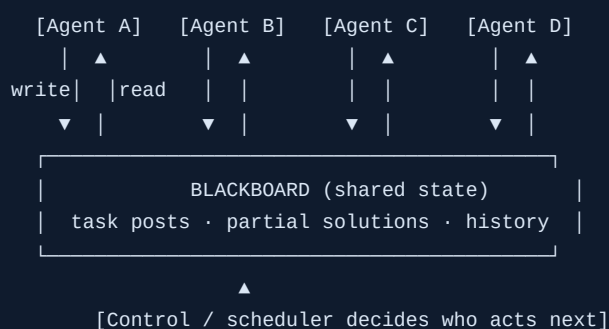
When to use. Many specialized agents that group naturally into domains, and you need a clear, **auditable** control structure. LangGraph’s framing: a supervisor tree is “easier to reason about — one routing node, clear control flow, every decision visible in traces.” Choose it when control and observability matter more than raw speed.

Failure modes.

- Supervisor is a **bottleneck and single point of failure**; each hop adds an LLM call (latency + cost).
- Deep hierarchies **amplify latency** (every level = a round-trip).
- The top supervisor can **mis-route between teams**, compounding routing error across levels.

7. Blackboard / shared-state

What it is. Agents coordinate **indirectly through a shared memory medium — the “blackboard”** — rather than messaging each other directly. A control agent **posts a request** describing the task/info needed; **any subordinate agent watching the board independently decides whether it can contribute**. The board holds public + private spaces with all messages, intermediate inferences, and interaction history.



When to use. Open-ended problems with **dynamic agent selection** (you don’t know in advance which specialist is relevant); information-discovery / data-science tasks; **token-sensitive settings** — the board keeps “compact, high-salience messages,” reducing inference cost and propagating info efficiently. Supports robust consensus mechanisms.

Failure modes.

- The **control/scheduling policy** (who writes next) is the hard part — poor control → thrashing or stalls.

- Shared state can grow unboundedly (history bloat) → context pressure.
- **Concurrency / write-contention** and consistency issues on the shared store.
- Harder to trace causality than a supervisor tree (coordination is indirect).

8. Network / handoff (peer-to-peer swarm)

What it is. A **decentralized** topology — agents **hand off control directly to one another** based on specialization, with **no central orchestrator**. “Network” = many-to-many (any agent can call any other). A **swarm** remembers which agent was last active, so the conversation **resumes with that agent** next turn.

- **LangGraph mechanism:** handoff via a `Command` object — `Command(goto=<agent>, graph=Command.PARENT)` navigates to a different node in the parent graph; state travels along.
- **OpenAI Agents SDK mechanism:** primitives are **Agents** (instructions + tools) and **handoffs** (a function returning another Agent). A triage agent routes to a specialist who **becomes the active agent** and replies directly. (The experimental *Swarm* was the predecessor; the production successor is the **OpenAI Agents SDK**, released March 2025, adding guardrails, tracing, sessions.)

```
[Agent A] ⇌ [Agent B]
  ⌞   ×   ⌞           any agent can hand off to any other;
[Agent C] ⇌ [Agent D]    no central coordinator
```

When to use. When the specialist should **answer the user directly** without a manager narrating results, and you want focused per-agent prompts. Swarms are “faster — no intermediary, direct agent-to-agent handoffs, fewer LLM calls” than supervisor trees.

Manager (agents-as-tools) vs. handoffs — OpenAI’s explicit contrast:

- **Manager / agents-as-tools:** a central manager calls specialists via `Agent.as_tool()`, **keeps control**, owns the final answer. (≈ centralized orchestrator-worker.)
- **Handoffs:** decentralized; control **transfers** to the specialist for that turn. (≈ swarm/network.)
- **Hybrid:** a triage agent can hand off to a specialist that simultaneously calls other agents as tools.

Failure modes.

- **No global view** — no agent sees the whole picture; hard to enforce overall goals.
- **Handoff loops / ping-ponging** between agents.
- Harder to trace and debug than centralized control.
- Routing errors propagate with no supervisor to catch them.

9. Debate & ensemble voting

What it is. Multiple agents **independently propose answers, then critique each other's reasoning across rounds to reach consensus** ("society of minds"). Ensemble voting is the lighter cousin: run multiple models/paths and **aggregate by majority/vote**. Debate induces an ensemble-like effect — diverse reasoning paths explored in parallel and **cross-verified**.

```
Round 1: [A answer] [B answer] [C answer]
          | exchange + critique |
Round 2: [A revises] [B revises] [C revises]
          | ... repeat ... |
          → converge → [Aggregate / Judge] → Output
```

When to use. Reasoning-heavy tasks (math, factual QA) where cross-verification reduces hallucination; LLM-as-judge ensembles where robustness matters.

Failure modes (2024–2026 research, treat as caveats).

- **"Problem drift"** — debates lose focus and wander off the original question over rounds (arXiv 2502.19559).
- **Cost/efficiency:** rounds × agents = heavy token spend. *EMS: Majority-then-Stopping* (arXiv 2604.02863) and adaptive stability detection (arXiv 2510.12697) exist specifically to cut voting cost and decide when to halt.
- **Not always worth it:** "Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking-Token Budgets" (arXiv 2604.02460) — at a fixed token budget, a single agent can beat debate. **Key caveat: gains may come from extra compute, not the topology itself.**
- Agents can converge on a **confidently-wrong consensus**.

10. Map-reduce over large corpora

What it is. Distributed map-reduce adapted for content that exceeds the context window. **Split** the corpus into token-bounded chunks → **Map:** apply the LLM independently to each chunk (summarize/extract) → **Reduce:** combine the intermediate outputs, recursively feeding them back until a single consolidated result remains.

```
Doc —split—┐ chunk1 → [LLM map] → s1 ┐
              │ chunk2 → [LLM map] → s2 ┤ [LLM reduce] → (recurse if needed) → Final
              └ chunk3 → [LLM map] → s3 ┘
```

Contrast set (long-content strategies).

- **Stuffing** — everything in one prompt; only if it fits.
- **Map-reduce** — parallel, scalable.
- **Refine** — sequential; carry a running summary chunk by chunk. Better continuity, less parallelism.

When to use. Corpus far exceeds the context window; chunks are largely independent; you want **parallel** throughput. Summarization, large-scale extraction/classification over many docs.

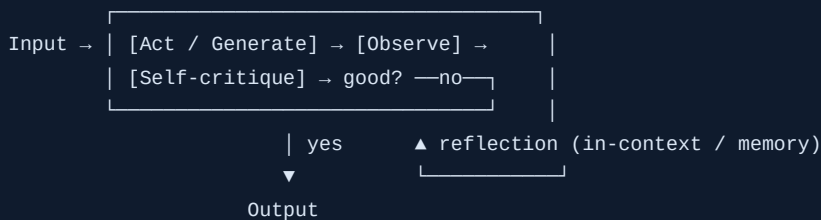
Failure modes.

- **Loss of global/cross-chunk context** — the map step can't see other chunks (cross-document entities/themes lost).
- Reduce step can itself overflow if there are too many intermediate summaries → needs **recursive/hierarchical reduce**.
- **Information dilution** compounding across reduce levels.
- Inconsistent map outputs are hard to merge cleanly.

11. Loop-until-done / reflection (ReAct, Self-Refine, Reflexion)

What it is. A single agent **iterates** — act, observe, **self-critique**, **refine** — until a quality threshold or stop condition is met. Family members:

- **ReAct** — interleaves reasoning (chain-of-thought) with acting (tool calls); improves interpretability and planning.
- **Reflection / Self-Refine** — the model is both writer and editor: critiques its own output and revises across loops, **no extra training**.
- **Reflexion** — extends ReAct: the agent decides whether to stop, **reflects on its entire trajectory**, stores **verbal reflections in persistent memory**, and **restarts** with that critique in-context. Learns across **multiple trials**.



Distinction vs. \$5 evaluator-optimizer: here **the same agent** critiques itself, often with persistent cross-trial memory; evaluator-optimizer uses a **separate** critic.

When to use. Tasks with **verifiable feedback** (tests pass/fail, tool errors, rubric); the agent benefits from learning from its own mistakes within or across attempts; iterative quality improvement on a single deliverable.

Failure modes.

- **Infinite / non-terminating loops** without a hard iteration cap.
- Self-critique is unreliable — the model may not recognize its own errors (no external ground truth).
- Reflection memory accumulates **noise**; cost grows per trial.

- Can **entrench an initial wrong approach** if reflections reinforce a bad frame.

12. Comparison table

Pattern	Parallelism	Determinism	Relative cost	Best for
Prompt chaining / pipeline	None (sequential)	High (fixed path)	Low	Fixed, decomposable tasks; accuracy > latency
Routing / dispatch	None (one branch)	High (after classify)	Low	Distinct input categories; cost-tiering models
Parallelization — sectioning	High (fixed branches)	High (predefined)	Medium (N×)	Independent subtasks; speed; guardrails
Parallelization — voting	High (same task ×N)	Medium (aggregation)	Medium–High (N×)	Confidence via multiple perspectives
Orchestrator-worker	High (dynamic count)	Low (LLM decides)	High (~15× chat)	Open-ended subtasks; breadth-first research; multi-file coding
Hierarchical / supervisor	Medium–High (per team)	Medium (visible flow)	High (hop per level)	Many grouped specialists; auditable control
Blackboard / shared-state	High (opportunistic)	Low–Medium (scheduler)	Low–Medium (token-efficient)	Dynamic agent selection; info discovery; token-sensitive
Network / handoff (swarm)	Medium (one active)	Low (decentralized)	Low–Medium (fewer calls)	Specialist answers directly; conversational routing
Debate / ensemble voting	High (agents parallel)	Low (consensus)	High (rounds × agents)	Reasoning/factual tasks; hallucination reduction*
Map-reduce over corpora	Very high (map)	High (structured)	Medium–High (per-chunk)	Content exceeding context window; summarization at scale
Loop-until-done / reflection	None (single loop)	Low (iterates)	Medium–High (per trial)	Verifiable-feedback tasks; iterative self-improvement

* *may not beat a single agent at equal token budget — see §9.*

Reading the table. Determinism falls and cost rises as you move from code-orchestrated (top) to LLM-orchestrated (bottom). Anthropic’s rule of thumb threads through every row: **use the simplest pattern that works; add agency only when simpler approaches demonstrably fail; pay for high parallelism/cost only on high-value tasks.**

13. Cross-cutting themes

- **Determinism ↔ flexibility is the master axis** (Anthropic workflows-vs-agents; OpenAI code-vs-LLM orchestration). Choose your spot deliberately.
 - **Cost scales with autonomy and parallelism:** agents $\approx 4\times$ chat tokens; multi-agent $\approx 15\times$ chat tokens. Multi-agent is “economically viable” only on high-value tasks.
 - **Token budget may explain the gains, not the topology:** 80% of performance variance traces to token usage; a single agent can match multi-agent at equal budget on some reasoning tasks. Don’t assume more agents = better.
 - **Observability is first-class:** non-determinism makes multi-agent debugging hard → full production tracing required. Supervisor trees are often chosen precisely because “every decision is visible in traces.”
 - **Coordination failure modes recur everywhere:** duplicate work, over-spawning, drift, loops, lost shared context. Standard mitigations: detailed delegation, complexity-scaled effort rules, distinct tool descriptions, explicit stopping criteria.
 - **Centralized vs. decentralized is the topology fork:** orchestrator / supervisor / manager (control + observability, but a bottleneck) vs. swarm / network / blackboard (speed + resilience, but harder to trace).
-

Sources

- Building Effective AI Agents — Anthropic — <https://www.anthropic.com/engineering/building-effective-agents>
- How we built our multi-agent research system — Anthropic — <https://www.anthropic.com/engineering/built-multi-agent-research-system>
- Multi-agent orchestration (manager vs. handoffs; LLM vs. code orchestration) — OpenAI Agents SDK — https://openai.github.io/openai-agents-python/multi_agent/
- Agents (primitives: instructions, tools, handoffs) — OpenAI Agents SDK — <https://openai.github.io/openai-agents-python/agents/>
- OpenAI Swarm (educational multi-agent framework) — <https://github.com/openai/swarm>
- LangGraph Multi-Agent Supervisor (reference) — <https://reference.langchain.com/python/langgraph-supervisor>
- langgraph-supervisor-py (supervisor pattern) — <https://github.com/langchain-ai/langgraph-supervisor-py>
- langgraph-swarm-py (swarm/handoff pattern, Command/goto) — <https://github.com/langchain-ai/langgraph-swarm-py>
- Multi-Agent Orchestration in LangGraph: Supervisor vs Swarm — DEV — https://dev.to/focused_dot_io/multi-agent-orchestration-in-langgraph-supervisor-vs-swarm-tradeoffs-and-architecture-1b7e

- Swarm vs. Supervisor: Multi-Agent Architecture Guide — Augment Code — <https://www.augmentcode.com/guides/swarm-vs-supervisor>
- LLM-based Multi-Agent Blackboard System for Information Discovery in Data Science — arXiv 2510.01285 — <https://arxiv.org/html/2510.01285v1>
- Collaborative Problem-Solving in Multi-Agent Systems with the Blackboard Architecture — <https://notes.muthu.co/2025/10/collaborative-problem-solving-in-multi-agent-systems-with-the-blackboard-architecture/>
- Scaling Document Summarization with LLMs: Stuffing, Map-Reduce, and Refine — Medium — <https://medium.com/@sonimegha1602/scaling-document-summarization-with-llms-stuffing-map-reduce-and-refine-a8a468d479c3>
- Summarizing Too Big for Context with MapReduce and LLMs — Google Cloud Community — <https://medium.com/google-cloud/summarizing-too-big-for-context-with-mapreduce-and-llms-6d2acc7a2ed0>
- Reflexion — Prompt Engineering Guide — <https://www.promptingguide.ai/techniques/reflexion>
- Reflection Agent Pattern — Agent Patterns docs — <https://agent-patterns.readthedocs.io/en/stable/patterns/reflection.html>
- Reflexion Agent Pattern — Agent Patterns docs — <https://agent-patterns.readthedocs.io/en/stable/patterns/reflexion.html>
- How Do Agents Learn from Their Own Mistakes? The Role of Reflection in AI — Hugging Face — <https://huggingface.co/blog/Kseniase/reflection>
- Stay Focused: Problem Drift in Multi-Agent Debate — arXiv 2502.19559 — <https://arxiv.org/pdf/2502.19559>
- EMS: Multi-Agent Voting via Efficient Majority-then-Stopping — arXiv 2604.02863 — <https://arxiv.org/pdf/2604.02863>
- Multi-Agent Debate for LLM Judges with Adaptive Stability Detection — arXiv 2510.12697 — <https://arxiv.org/html/2510.12697v1>
- Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets — arXiv 2604.02460 — <https://arxiv.org/html/2604.02460v1>
- Cloudflare Agents — Anthropic patterns guide — <https://github.com/cloudflare/agents/blob/main/guides/anthropic-patterns/README.md>

Communication, Coordination & Interop Protocols (MCP, A2A, handoffs)

Source: [readme/research/agent-orchestration/AGENT_ORCHESTRATION_03_COMMUNICATION_PROTOCOLS_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document explains how AI “agents” (software helpers that do tasks on their own) talk to each other and to outside tools. When you build a system with multiple AI agents working together, they need clear rules for passing work, sharing information, and connecting to databases or services — this doc covers all those rules. Understanding this helps you build AI systems that don’t break, waste money, or produce wrong results.

The main parts explained simply:

- **Two ways agents share information** — Agents can either send direct messages to each other (like email), or write to a shared “whiteboard” that everyone reads. Direct messages give independence but create a lot of back-and-forth noise. A shared whiteboard is cleaner but needs careful access rules to avoid two agents overwriting each other at the same time.
- **Handoffs (passing the job to another agent)** — When one AI agent finishes its part and hands the task to another, this is called a handoff. Both OpenAI and LangGraph (two popular AI frameworks) say the same thing: always use a formal handoff step, never let agents call each other directly. This keeps it clear who is in charge at any moment.
- **Breaking big tasks into smaller pieces** — Anthropic’s research system shows how a “lead” AI agent splits a big question into many smaller questions, sends each to a separate “worker” agent running in parallel, then collects and combines all the answers. This can cut research time by up to 90%, but costs about 15 times more than a simple chat — so it’s only worth it for genuinely complex, high-value tasks.
- **MCP (Model Context Protocol)** — A standard created by Anthropic in 2024 that gives AI agents a single, consistent way to connect to tools and data sources (databases, files, apps). Instead of building a custom connector for every combination, you build one MCP connector and it works everywhere.
- **A2A (Agent2Agent Protocol)** — A standard created by Google in 2025 that lets AI agents made by different companies discover and delegate tasks to each other. Each agent publishes a public “Agent Card” saying what it can do, without revealing how it works inside.
- **Tool calling** — The basic action an AI agent takes when it needs to do something (run a search, query a database, or pass work to another agent). In practice, handing off to another agent looks exactly the same as calling a tool — the AI just picks the right one.
- **Structured outputs (enforcing message shape)** — When agents send results to each other, those results must follow a strict format (like a form with fixed fields), not free-form text. Without this enforcement, messages fail to parse correctly about 8–15% of the time. With proper schema enforcement, failure drops below 0.2%.
- **Context and state pitfalls** — Jamming too much information into one agent’s memory degrades its performance. Errors from one agent can also ripple out and break the whole system. The fix: give each agent only the information it needs for its specific job, and save the overall plan to memory before splitting work across many agents.
- **MCP vs A2A — they work together, not against each other** — MCP connects each agent to its own tools and data (vertical). A2A connects agents to other agents (horizontal). A well-built multi-

agent system uses both at the same time.

What to do with this: If you are building or overseeing an AI system with multiple agents, use MCP to connect each agent to its tools, and A2A to let agents coordinate with each other. Always enforce a strict message format between agents — free-form text breaks too often. Reserve expensive multi-agent setups for tasks that are genuinely complex and can run in parallel; for simple tasks, a single agent is cheaper and good enough.

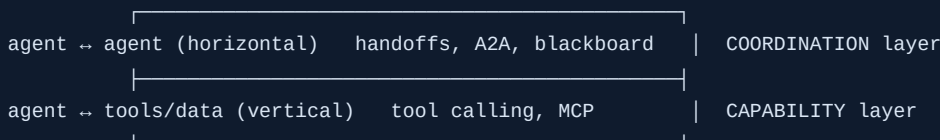
How agents pass control, share state, and talk to tools and to each other — message passing vs. blackboards, handoff semantics, MCP, A2A, and structured-output reliability.

Research/reference doc · 2026-06-16 · part of the Agent Orchestration suite

0. Orientation

A multi-agent system (MAS) is only as good as the plumbing between its agents. This doc covers the **wire-level and coordination-level questions**: how do agents exchange information, who is “driving” at any moment, how is a big task split and re-assembled, and what open protocols (MCP, A2A) standardize agent ↔ tool and agent ↔ agent communication.

Two mental layers run through everything below:



Keep that split in mind — MCP lives on the vertical layer, A2A on the horizontal one, and they are complementary (§9).

1. How agents communicate: message passing vs. shared memory (blackboard)

Two foundational coordination paradigms underlie all MAS.

Message passing (peer-to-peer / conversational)

Agents communicate **directly**, negotiating tasks and sharing information through conversational exchanges — dialogue histories and task-specific protocols. Early MAS frameworks relied on **isolated local memories plus explicit message passing**.

- **Pros:** flexible; agents keep their own state, so they don’t interfere with each other.

- **Cons:** every handoff adds overhead. As team size and task horizon grow, you get **redundancy, fragmented context, and high communication overhead**.

Blackboard architecture (shared memory)

A **globally accessible shared knowledge base** (“blackboard”) is the sole memory and message-passing hub. Agents with different roles **independently monitor** the shared state and contribute when they can add value.

- Each agent typically reads the *entire* blackboard before acting; all outputs are posted back as new blackboard entries.
- Agent selection **emerges from current content** — repeated execution rounds run until consensus.
- **Pros:** transparency; far less message traffic; loose coupling (agents post intermediate results and read others’ contributions without tight coupling).
- **Cons:** needs **strong versioning and access control**; **consistency and concurrency** problems when multiple agents read/modify the same state at once (requires locks, transactions, or optimistic concurrency control).

The core trade-off

	Message passing	Blackboard / shared memory
Coupling	Loose (decoupled agents)	Loose, but via shared substrate
Coordination	Explicit, per-message	Implicit (observe + update state)
Message traffic	High, grows with team size	Low
Main failure mode	Context fragmentation, overhead	Concurrency / consistency bugs
Visibility	Hard to see the whole picture	High transparency

Shared state **reduces explicit message passing** but trades it for concurrency complexity. Message passing keeps agents independent but multiplies overhead.

Practical hybrid: LangGraph state object

LangGraph is built around a **state object** with a defined **state schema**, accessible at each agent step. The state behaves like a “collaborative whiteboard” (a blackboard) — all nodes can read/modify shared info — while supporting both **complete attribute override** and **additive (append) updates**, keeping data consistent. This blends blackboard-style shared memory with explicit graph-based control flow, which is why it’s the default choice for production graphs.

2. Handoff semantics: control transfer vs. context transfer

A **handoff** is the primitive by which one agent delegates to another. Two distinct things can move during a handoff — keep them separate in your head:

- **Control transfer** — *which* agent is active / driving the conversation.
- **Context transfer** — *what* information (history, task data) the receiving agent gets.

OpenAI Agents SDK (production successor to Swarm)

- A **handoff** lets an agent delegate a task to another agent — useful when agents specialize.
- Handoffs **transfer control entirely**: the receiving agent **becomes the active agent** and responds directly to the user.
- Crucially, handoffs are **represented to the LLM as tools**. A handoff to a “Refund Agent” surfaces as a tool named `transfer_to_refund_agent`. This unifies handoffs with the tool-calling mechanism (see §6).
- History: **Swarm** (open-source, Oct 2024) was the lightweight prototype; the **Agents SDK** (Mar 2025) is its production evolution. Swarm is no longer maintained.

LangGraph

- Same conceptual shape: agents **transfer control through an explicit handoff primitive instead of calling each other directly**.
- Implemented via `Command(goto=...)` for dynamic flow — route to a named node, rather than OpenAI’s “handoff-as-tool” framing.
- `langgraph-swarm-py` provides swarm-style handoff patterns; the `interrupt` primitive supports cleaner human-in-the-loop pauses.

Design principle: explicit handoff, not direct calls

Both major frameworks converge on the same rule: **use an explicit handoff primitive, never direct agent-to-agent function calls**. This enforces **ownership boundaries** — clear control-transfer points instead of tangled mutual calls that nobody can reason about.

```
[Triage Agent] --transfer_to_refund_agent--> [Refund Agent] (control moves; triage goes idle)
      ^                                     |
      |____ handoff back _____|
```

When to use handoffs: distinct specializations, clear “this is now someone else’s job” boundaries.

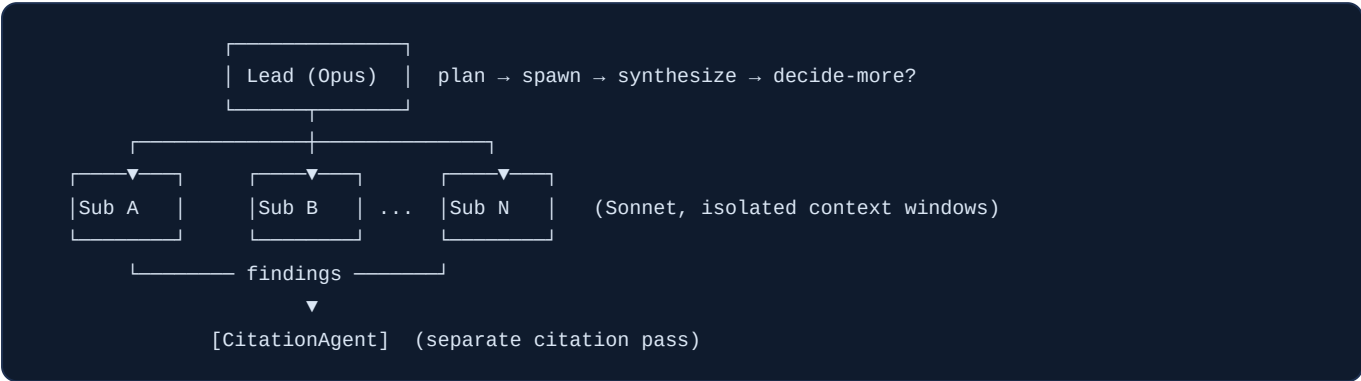
When NOT to: tasks needing tight back-and-forth or shared working context — that’s a sub-agent/tool call within one owner, not a control transfer.

3. Task decomposition & result aggregation (Anthropic multi-agent research system)

The canonical production case study is Anthropic’s Research feature — an **orchestrator-worker (lead + subagent)** pattern.

Architecture: orchestrator-worker (hierarchical)

- A **lead agent (Lead Researcher)** analyzes the query, develops strategy, and **spawns multiple subagents** to explore different aspects in parallel.
- Production config: **Claude Opus 4 as lead, Sonnet 4 as subagents**.



Task decomposition

The lead breaks complex queries into subtasks with **explicit guidance**. Each subagent needs **an objective, an output format, guidance on which tools/sources to use, and clear task boundaries**. Without this, subagents **duplicate effort** — early versions had multiple agents investigating identical topics from vague directives.

Scaling rules embedded in prompts to control effort:

Query type	Subagents	Tool calls each
Simple fact-finding	1	3–10
Comparisons	2–4	10–15
Complex research	10+	divided responsibilities

Context & result aggregation

- **Isolated context windows:** each subagent runs in its own context window — enabling parallel reasoning and scaling past a single agent’s limit. Performance gains are strongly linked to spreading reasoning across **multiple independent context windows**.
- **Persistent memory:** the lead **saves its research plan to memory** before launching subagents, so it survives nearing the **200,000-token limit**.
- **Synthesis:** subagents return findings to the lead, which synthesizes and decides whether more research is needed.
- **Separate citation pass:** a dedicated **CitationAgent** processes documents afterward to attribute claims to sources.
- **Parallel tool calling:** subagents issue 3+ tool calls simultaneously; the lead spins up 3–5 subagents in parallel — cutting research time by **up to 90%** on complex queries.

Performance & cost

- The multi-agent system **outperformed single-agent Claude Opus 4 by 90.2%** on internal research evals (especially breadth-first / parallelizable queries).
- **Token economics:** multi-agent uses **~15× the tokens** of an ordinary chat (chat agents already use ~4×). **Token usage alone explains ~80% of performance variance** — so this is only viable for high-value tasks.

Failure modes found (and fixes)

- Spawning **50 subagents for a trivial query**; scouring the web endlessly for sources that don't exist. Fixed with explicit scaling rules, teaching delegation granularity, and using extended thinking as a “controllable scratchpad”.
- **Prompt engineering was the single biggest lever** — small phrasing changes separated efficient research from wasted effort.

When NOT to use multi-agent: domains needing **shared context across agents** or **heavy interdependencies** are poor fits. “Most coding tasks involve fewer truly parallelizable tasks than research.” Synchronous execution (lead waits for subagent batches) creates bottlenecks but simplifies coordination — a deliberate trade.

4. Model Context Protocol (MCP)

Creator / date: Anthropic, **November 2024**. Open standard, open-source.

What it standardizes

A **universal, open standard for connecting AI systems to data sources**, replacing fragmented one-off integrations with a single protocol. It enables **secure, two-way connections** between data/tools and AI applications, and solves the **N×M integration problem**: instead of a custom connector for every model × every data source, developers **build against one protocol**. The framing: models were “trapped behind information silos and legacy systems.”

Client-server model

- **Two tiers:** developers either **expose data via MCP servers**, or **build AI applications (MCP clients)** that connect to them. The AI app is the client; servers deliver external data/capabilities.
- **Discovery:** a protocol handshake — the client connects and the server **registers/advertises its capabilities**.

Three core primitives

Primitive	Controlled by	What it is
Tools	Model-controlled	Executable functions the model can invoke (actions, queries)
Resources	App-controlled	Data/content the app exposes (files, records)
Prompts	User-controlled	Reusable prompt templates the user can invoke

Transport & technical

- Built on **JSON-RPC 2.0**; re-uses message-flow ideas from the **Language Server Protocol (LSP)**.
- Transports: **stdio** (local) and **SSE / HTTP** (remote).
- SDKs provided (e.g. JS `@modelcontextprotocol/sdk`). Anthropic shipped pre-built servers for Google Drive, Slack, GitHub, Git, Postgres, Puppeteer; Claude Desktop supports local servers.
- Latest referenced spec revision: **2025-11-25**.
- **Code execution with MCP** is an emerging efficiency pattern — agents call MCP tools via generated code rather than many slow round-trips.

Adoption

From **100,000** → **97 million** monthly SDK downloads in 16 months; **78% of enterprise AI teams** report at least one MCP-backed agent in production.

5. Agent2Agent (A2A) protocol

Creator / date: Google, announced **April 2025**, with **50+ partners**. Open standard.

Purpose

Lets AI agents **built by different vendors discover each other, delegate tasks, and coordinate** across enterprise systems — interoperability **without shared direct access** to each other’s resources. Agents stay **opaque** about their internal state and tools.

Agent Cards (discovery)

Every A2A agent publishes an **Agent Card** — a JSON metadata document at the well-known URL `/.well-known/agent-card.json`. It describes:

- identity (name/description), version, service endpoint
- supported modalities/interfaces
- authentication/security schemes
- capability flags (streaming, push notifications, extended cards)

Cards may be **signed and cached**. Capabilities are advertised **without exposing implementation details**.

Roles (asymmetric)

- **A2A Client (client agent)**: initiates requests on behalf of users.
- **A2A Server (remote agent)**: processes tasks, publishes capabilities via its Agent Card.

This asymmetry is what lets agents collaborate while staying opaque.

Task lifecycle

The **Task** is the fundamental unit of work and has a unique ID. States:

```
SUBMITTED → WORKING → COMPLETED (terminal)
      |      |      |
      |      |      |→ FAILED (terminal)
      |      |      |→ CANCELED (terminal, user-initiated)
      |      |      |→ REJECTED (terminal, agent declined)
      |      |→ INPUT_REQUIRED (interrupted, needs user input)
      |→ AUTH_REQUIRED (interrupted, needs auth)
```

Plus **unknown**. **Terminal states prevent further messages.**

Messages & Artifacts

- **Message** = one unit of communication, with a **role** (user or agent) and **Parts** (text, files, or structured data).
- **Artifact** = a task **output**, composed of Parts.
- Deliberate separation: **communication (Messages) vs. results (Artifacts)**; task history preserves the full interaction record.

Transport & technical

- Three bindings: **JSON-RPC 2.0**, **gRPC**, **HTTP/REST**.
- Updates via **Server-Sent Events** (real-time streaming), **polling** (Get Task), and **async push notifications to webhooks**. Clients can manage one or many concurrent streams per task.
- Enterprise security: **OAuth 2.0**, **API keys**, **mTLS**.
- Capability pillars: discovery, task management, message-based collaboration, artifact handling, UX negotiation.

6. Function/tool calling as a coordination primitive

Function/tool calling lets a model **invoke predefined functions with structured parameters** — the base mechanism for an agent to act on the world. It also doubles as a **coordination primitive**:

- In the OpenAI Agents SDK, **handoffs are exposed as tools** (`transfer_to_<agent>`), so delegating to another agent is mechanically identical to calling a tool.
- MCP **tools** are likewise model-controlled functions.

The unification: tool calling is the substrate for **both** agent → tool access (MCP) **and** agent → agent delegation (handoffs / A2A tasks). The model decides *which* tool/handoff to invoke; the framework routes control accordingly. This is why “give the agent a tool” and “give the agent a colleague” look the same at the API level.

7. Structured outputs / schemas for reliable inter-agent messages

Reliable coordination requires messages that conform to a schema so the receiver can parse them **deterministically**. Free-form text between agents is a recipe for fragile parsing and silent breakage.

State of the art (2025)

Provider / mode	Schema compliance
OpenAI Structured Outputs (full JSON Schema attached)	99.9%
Anthropic Structured Outputs (beta)	99.8% (tool use), <0.2% failure, ~150–300 token overhead
Gemini schema	~99.7%
Plain JSON mode, no schema enforcement	fails 8–15% of the time

- **Anthropic Structured Outputs** (public beta, late 2025) require responses to strictly conform to a JSON schema or a tool spec, activated via the `structured-outputs-2025-11-13` beta header.
- The 8–15% failure rate of unenforced JSON is precisely why schema enforcement matters for inter-agent reliability.

Two distinct mechanisms

- **Structured outputs** = guarantee the *shape of the model's response* matches a schema → good for typed inter-agent messages/artifacts.
- **Function/tool calling** = model selects a predefined function and supplies structured parameters → good for actions.

Both feed the same goal: cleaner, stricter, machine-parseable output that wires into downstream software/agents without brittle parsing.

Relevance to protocols: A2A's **Parts** (text/file/structured-data) and MCP's structured JSON-RPC payloads both depend on this reliability. Typed, schema-validated exchange is what makes artifact handling between agents dependable rather than hopeful.

8. State & context-passing pitfalls

From LangChain/LangGraph “context engineering” guidance and MAS analyses.

The failure modes

- **Context window pollution** — cramming too many tools/instructions into one prompt **degrades performance**, and the degradation is **hard to predict or reproduce**. In a flat-context orchestrator, concurrent agents’ partial outputs and domain state **compete for one context window**.
- **Error propagation / blast radius** — **without isolation boundaries, one bad tool call or hallucination can derail the entire workflow**. Isolation limits the blast radius of a single agent's error.
- **State-management complexity** — coordination adds consistency/concurrency overhead (the same blackboard problem from §1).

Mitigations (LangGraph-centric)

- **Context isolation** — store tool-call context in dedicated **state fields**, hidden from the LLM until needed; **split context across sub-agents** so each owns its sub-task with its own tools/instructions/context window (mirrors Anthropic’s isolated context windows in §3).
- **Defined state schema** — the shared state object enforces structure and supports both override and additive updates while staying consistent.
- **Newer primitives** — `Command` (dynamic flow / `goto`), `interrupt` (human-in-the-loop), **semantic search** for long-term memory, **cross-thread memory** for state across conversations.
- **Persistent plan storage** (Anthropic) — save the plan to memory *before* spawning workers, so context-window overflow doesn’t wipe the orchestration state.

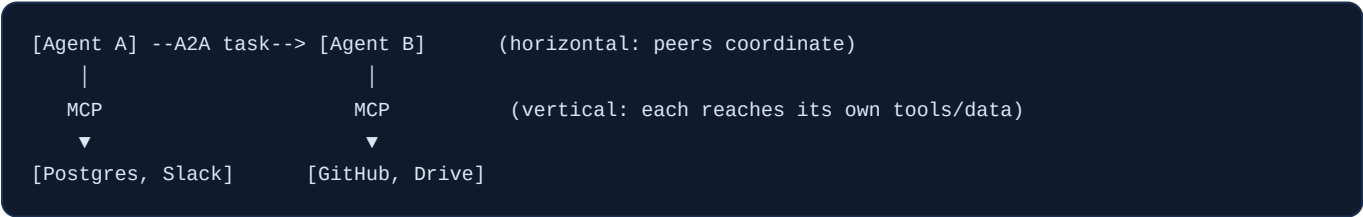
Rule of thumb: the receiving agent should get *exactly* the context it needs for its sub-task — no more (pollution, blast radius) and no less (duplicated work, hallucinated gaps). Decide per-handoff what context transfers.

9. MCP vs A2A — comparison

Dimension	MCP (Model Context Protocol)	A2A (Agent2Agent)
Core question	“How does an agent talk to tools/data ?”	“How do agents talk to each other ?”
Creator / date	Anthropic, Nov 2024	Google + 50+ partners, Apr 2025
Layer / direction	Vertical — agent ↔ tools & data	Horizontal — agent ↔ agent (peers)
Primary unit	Tools, Resources, Prompts	Task (with lifecycle), Messages, Artifacts
Discovery	Handshake; server registers capabilities	Agent Card at <code>/.well-known/agent-card.json</code>
Transport	JSON-RPC 2.0 over stdio / SSE / HTTP (LSP-inspired)	JSON-RPC 2.0 / gRPC / HTTP+REST; SSE, webhooks, polling
Opacity model	Server exposes concrete tools/data to client	Agents stay opaque — declared capabilities only
Security	Per server/transport	OAuth 2.0, API keys, mTLS built in
The “other” side	Non-agent resources	A peer agent that owns the task lifecycle
Analogy	Powers an agent internally with context & tools	Connects agents externally for collaboration

Relationship: complementary, not competing

They operate at **different layers of the same stack**. A production multi-agent system typically uses **MCP for each agent’s data/tool access** and **A2A for coordination between agents**. Most enterprise deployments implement **both**, applying least-privilege/auth at each layer.



10. Practical takeaways

- **Pick a coordination model deliberately.** Shared-state/blackboard (LangGraph state object) for transparency and low traffic; message passing for decoupling. Most production graphs use the hybrid: a typed shared state with explicit control flow.
- **Make handoffs explicit primitives**, never direct agent-to-agent calls — this is the one rule both OpenAI and LangGraph agree on, and it’s what keeps ownership boundaries clean.
- **Decompose with explicit per-subagent contracts** (objective, output format, tools, boundaries). Vague delegation causes duplicated work and runaway subagent counts.

- **Budget tokens like money** — multi-agent is ~15× the cost of chat; reserve it for high-value, parallelizable, breadth-first work.
 - **Enforce schemas on every inter-agent message.** Unenforced JSON fails 8–15% of the time; structured outputs push that under 0.2%.
 - **Isolate context per agent** to cap the blast radius of errors and avoid context pollution; persist the orchestration plan before spawning workers.
 - **MCP and A2A are not rivals** — wire MCP under each agent and A2A between agents.
-

Sources

- Introducing the Model Context Protocol — <https://www.anthropic.com/news/model-context-protocol>
- MCP Specification (2025-11-25) — <https://modelcontextprotocol.io/specification/2025-11-25>
- Model Context Protocol (Wikipedia) — https://en.wikipedia.org/wiki/Model_Context_Protocol
- Code execution with MCP (Anthropic engineering) — <https://www.anthropic.com/engineering/code-execution-with-mcp>
- How we built our multi-agent research system (Anthropic engineering) — <https://www.anthropic.com/engineering/built-multi-agent-research-system>
- Agent2Agent (A2A) Protocol Specification — <https://a2a-protocol.org/latest/specification/>
- Announcing the Agent2Agent Protocol (Google Developers Blog) — <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>
- A2A vs. MCP (Descope) — <https://www.descope.com/blog/post/mcp-vs-a2a>
- MCP vs A2A Protocol: Architecture, Differences and When to Use (Atlan) — <https://atlan.com/learn/mcp/mcp-vs-a2a-protocol/>
- A2A vs MCP — How These AI Agent Protocols Actually Differ (DigitalOcean) — <https://www.digitalocean.com/community/tutorials/a2a-vs-mcp-ai-agent-protocols>
- Handoffs — OpenAI Agents SDK docs — <https://openai.github.io/openai-agents-python/handoffs/>
- langgraph-swarm-py (LangChain) — <https://github.com/langchain-ai/langgraph-swarm-py>
- Multi-Agent Handoff With Ownership Boundaries (dev.to) — <https://dev.to/gabrielanhaia/multi-agent-handoff-with-ownership-boundaries-nobody-crosses-nll>
- Context Engineering for Agents (LangChain blog) — <https://www.langchain.com/blog/context-engineering-for-agents>
- Multi-Agent Systems: Design Patterns and Orchestration (Tetrade) — <https://tetrade.io/learn/ai/multi-agent-systems>
- Multi-agent systems: Why coordinated AI beats going solo (Redis) — <https://redis.io/blog/multi-agent-systems-coordinated-ai/>
- bMAS: Blackboard LLM Multi-Agent System (EmergentMind) — <https://www.emergentmind.com/topics/blackboard-based-llm-multi-agent-system-bmas>

- Structured outputs — Claude API Docs — <https://platform.claude.com/docs/en/build-with-claude/structured-outputs>
- Anthropic boosts Claude API with Structured Outputs (Tessl) — <https://tessl.io/blog/anthropic-brings-structured-outputs-to-claude-developer-platform-making-api-responses-more-reliable/>
- Structured Output and JSON Mode Guide 2026 (TokenMix) — <https://tokenmix.ai/blog/structured-output-json-guide>
- From Glue-Code to Protocols: Critical Analysis of A2A and MCP (arXiv 2505.03864) — <https://arxiv.org/pdf/2505.03864>

Frameworks & SDKs Landscape

Source: [readme/research/agent-orchestration/AGENT_ORCHESTRATION_04_FRAMEWORKS_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a plain guide to the main “toolkits” (called frameworks) that developers use when building AI agents — software that can think through a task and take actions on its own. It matters because choosing the wrong toolkit wastes months of work, and many teams pick a complicated one when a simple approach would do the job fine.

The main parts explained simply:

- **Start simple — you may not need a framework at all** — Before picking any toolkit, ask whether a single AI call or a short, fixed sequence of steps can do the job. For most tasks, the answer is yes. Frameworks add hidden complexity that makes problems harder to find and fix.
- **Workflow vs. Agent** — A “workflow” follows a fixed path you draw in advance (like a flowchart). An “agent” decides its own next step as it goes. Use workflows when the task is predictable; use agents only when you truly cannot predict the path ahead.
- **LangGraph** — A toolkit where you map out every possible step and decision as a diagram (a “graph”). Best when you need full control, the ability to pause and resume long tasks, and a clear audit trail of what happened. The most powerful option, but also the hardest to learn.
- **CrewAI** — Lets you define AI agents as if they were team members with job titles (researcher, writer, reviewer). They work together automatically. Also has a “Flows” mode for more predictable, step-by-step control. Good for tasks that naturally split into team roles.
- **Microsoft AutoGen / AG2** — Agents that work by having a “conversation” with each other to solve a problem. Useful for code-writing tasks, but be cautious: the project has split into two competing versions and is merging with another Microsoft product — so the ground is shifting.
- **OpenAI Agents SDK** — A lightweight toolkit from OpenAI where one agent can hand off a task to another specialist agent (called “handoffs”). Includes built-in safety checks (guardrails) and logging. Easy to learn, but works best if you are already using OpenAI’s AI models.
- **Claude Agent SDK (Anthropic)** — The same engine that powers Claude Code (the AI coding assistant). Best for tasks involving files, code, or computer use. Has built-in safety controls (hooks) and lets you add custom tools. Only works with Claude models.
- **Google ADK + A2A** — Google’s toolkit for building agents, with a bonus: the A2A (“Agent2Agent”) standard lets agents built on *different* toolkits talk to each other. Useful if you need agents from different systems or vendors to cooperate.
- **Microsoft Semantic Kernel** — A corporate-focused toolkit designed for teams using Microsoft’s technology stack (.NET, Azure). Being merged with AutoGen into a single “Microsoft Agent Framework.” Best for large enterprises already inside the Microsoft ecosystem.
- **LlamaIndex** — A toolkit built around searching and reasoning over large collections of documents. Best when the agent’s main job is reading, finding, and summarizing information from big knowledge bases.
- **MCP vs. A2A protocols** — Two open standards that work together. MCP connects an agent to tools and data (what an agent can *do*). A2A connects agents to *other agents* across different systems. Think of MCP as giving an agent its hands, and A2A as letting it call a colleague.

What to do with this: Before choosing any toolkit, check whether a plain sequence of AI calls will do the job — it usually will. If you do need a framework, pick the lightest one that fits your situation using the comparison table in section 10, and always make sure you understand what it is doing underneath.

A field guide to the agent-orchestration frameworks and interop protocols of 2025–2026 — what each one actually does, and when (and when not) to reach for it.

Research/reference doc · 2026-06-16 · part of the Agent Orchestration suite

0. Read this first: do you even need a framework?

Anthropic’s own guidance (*Building Effective AI Agents*) is blunt: **start with direct API calls**. Many useful patterns are “a few lines of code.” Find the simplest viable solution and add complexity only when it *demonstrably* improves outcomes. For a large share of apps, a single optimized LLM call with retrieval and in-context examples is enough.

Why frameworks carry a tax:

- They **add abstraction layers that obscure the underlying prompts and responses**, making debugging harder.
- They make it tempting to add complexity when a simpler setup would do.
- “Incorrect assumptions about what’s under the hood are a common source of customer error.”

The practical rule: **if you use a framework, understand the code underneath it**, and don’t hesitate to drop back to basic components in production.

Workflows vs. Agents (the distinction everything hangs on)

	What it is	Use when
Workflow	LLMs + tools orchestrated through predefined code paths (you control the flow).	The task is well-defined and predictable.
Agent	An LLM dynamically directs its own process and tool usage .	The problem is open-ended and you can’t predict the path.

The base building block beneath both is the **Augmented LLM** — a model enhanced with **retrieval, tools, and memory**, generating its own queries and managing what to retain.

Five workflow patterns to build before reaching for an agent:

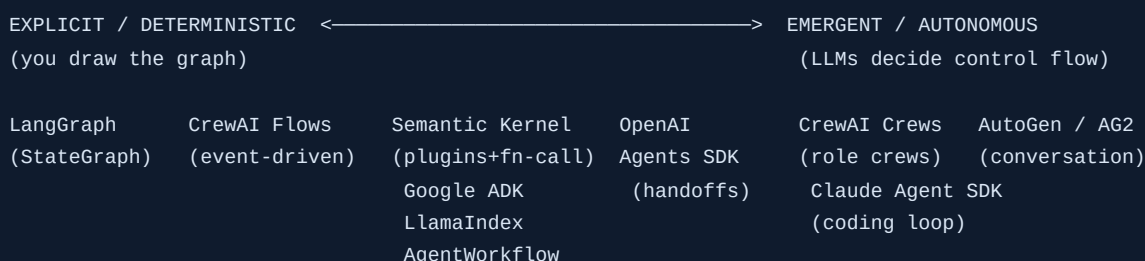
1. **Prompt chaining** — sequential calls, each on the prior output (trades latency for accuracy).
2. **Routing** — classify input, dispatch to a specialized handler.
3. **Parallelization** — *sectioning* (split into parallel subtasks) or *voting* (multiple attempts).
4. **Orchestrator-workers** — a central LLM decomposes a task and dispatches to workers.

5. **Evaluator-optimizer** — one LLM generates, another critiques and feeds back to refine.

Bottom line: Frameworks are *not* the starting point. Reach for one only when a real need appears — durable state, multi-agent routing, observability at scale, or org-wide standardization — and pick the lightest one that meets it.

1. The landscape, on two axes

The frameworks differ most along a **control spectrum** — how much the flow is drawn by you vs. decided by the LLM at runtime:



A second axis worth tracking: **single-vendor vs. provider-agnostic**, and **proprietary control loop vs. open protocol**. Two standards cut across *all* of them:

- **MCP** (Model Context Protocol) — connects an agent to **tools and data**.
- **A2A** (Agent2Agent) — connects an agent to **other agents** across vendors/frameworks.

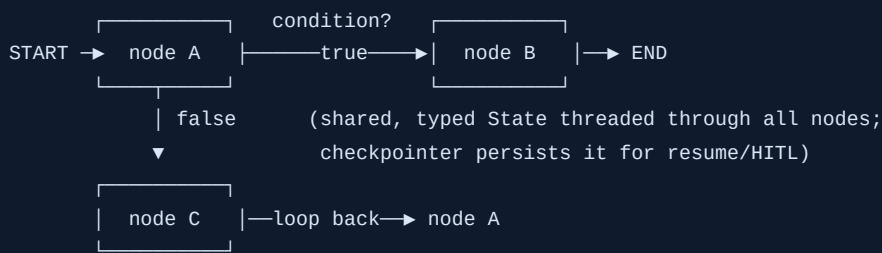
These are complementary, not competitors (see §11).

Naming hazard (2025–2026): This space renamed itself constantly. Swarm → OpenAI Agents SDK; `claude-code-sdk` → Claude Agent SDK; AutoGen 0.2 → AG2 (community fork) vs AutoGen 0.4 (Microsoft rewrite), both now consolidating with Semantic Kernel into the **Microsoft Agent Framework** (announced Oct 2025). Dates are stated explicitly throughout.

2. LangGraph — graph / state-machine

Model. Agents are a **directed graph** (`StateGraph`). Nodes = functions / tool calls / model calls / decision points; edges = control flow. A centralized, typed **shared state object** is threaded through every node, and **conditional edges** carry predicates over that state to choose the next node.

Primitives. Nodes, edges, conditional/branching edges, the State schema, **checkpointers** (durable persistence for long-running, resumable runs), loops/cycles, retries, human-in-the-loop interrupts, streaming, parallel execution.



Use it when	Avoid / reconsider when
You need explicit, auditable control over complex flows with loops, retries, branching, long-running/durable state , and multi-actor orchestration.	You want a quick prototype or simple linear flow; the team can't absorb graph/state-schema modeling.

Strengths: most explicit and controllable; first-class cycles (not just DAGs), durable memory, multi-agent topologies; production-grade observability when paired with LangSmith. **Weaknesses:** steepest conceptual ramp — you architect nodes, edges, and state schemas deliberately; heavier than a few API calls for simple tasks.

3. CrewAI — role-based crews (+ Flows)

Model. Two complementary modes:

- **Crews** — teams of autonomous agents, each defined by **role, goal, and backstory** (a metaphor that maps to how humans think about teams). Agents collaborate and decide autonomously, in a sequential or hierarchical process.
- **Flows** — **event-driven, production-oriented** workflows that chain crews with conditional logic, state, and event triggers for precise, deterministic control. Flows can embed crews as steps.

Primitives. Agent (role/goal/backstory), Task, Crew, Process (sequential/hierarchical), Tools; Flows add events, state, conditional routing.

Use it when	Avoid / reconsider when
Crews: work decomposes into human-team-like roles that collaborate (research → write → review).	You need strict, predictable control paths — use Flows or LangGraph instead.
Flows: you need event-driven, deterministic orchestration that can also embed autonomous crews.	A single autonomous crew already suffices (over-engineering risk).

Strengths: the most intuitive mental model → fast to prototype collaborative systems; Crews (autonomy) + Flows (determinism) give both adaptive problem-solving and auditable execution. **Weaknesses:** autonomous crews are harder to constrain than an explicit graph; the role-play abstraction can obscure exact control flow.

4. Microsoft AutoGen / AG2 — conversational agents

Model. Multi-agent conversation. Agents are customizable and “conversable,” combining LLMs, human input, and tools. Work happens through structured chat between agents (assistant + user-proxy + group-chat patterns).

Primitives. Conversable agents, group chat / chat managers, human-in-the-loop (`UserProxyAgent`), tool/code execution, conversation patterns.

Ecosystem timeline (this is the adoption risk):

Date	Event
2023	Original AutoGen research framework (Microsoft Research).
Jan 2025	AutoGen 0.4 — full architectural rewrite: modular components, async, scalable, built-in debugging/monitoring, TypeScript support, distributed agents.
2024–25	AG2 = community fork continuing AutoGen 0.2 , maintained by the original creators after they left Microsoft.
Oct 2025	Microsoft announces AutoGen + Semantic Kernel consolidating into a unified Microsoft Agent Framework — AutoGen’s orchestration + SK’s enterprise plugins/security/Azure. Official AutoGen → Agent Framework migration guide published.

Use it when	Avoid / reconsider when
You want conversational multi-agent problem-solving / code-execution loops and can track the ecosystem.	You need long-term stability <i>now</i> — watch the AutoGen + SK → Microsoft Agent Framework consolidation before committing.

Strengths: strong research lineage; flexible conversation patterns; good for emergent problem-solving and code-execution loops. **Weaknesses:** fragmented ecosystem (0.2 vs 0.4 vs AG2 vs forthcoming Agent Framework) — version/naming confusion is a genuine risk in this window; conversation-driven control is hard to constrain.

5. OpenAI Agents SDK — handoffs & guardrails, lightweight

History. Successor to **Swarm** (experimental/educational, Oct 2024). Released production-ready on **March 11, 2025**.

Model & primitives (from the official docs):

- Agents** — LLMs equipped with instructions and tools.
- Handoffs** — delegate/transfer control to another specialized agent without manually wiring routing or state.
- Guardrails** — input/output validation that runs before/after the model (block prompt injection at the boundary, enforce schema/policy).

Plus **Sessions** (persistent memory) and **Tracing** (every run emits a structured trace — which agents ran, which tools were called, which handoffs occurred; built-in tracing UI + exporters to third-party observability).

Design philosophy (quoted): “(1) Enough features to be worth using, but few enough primitives to make it quick to learn. (2) Works great out of the box, but you can customize exactly what happens.”

Provider support: OpenAI by default but **not locked in** — non-OpenAI providers via adapters (LiteLLM, Any-LLM).

Use it when	Avoid / reconsider when
You're on/near OpenAI models and want a lightweight layer with handoffs , guardrails , sessions, and built-in tracing , minimal abstraction.	You need a rich graph engine or heavy non-OpenAI orchestration.

Strengths: minimal learning curve; handoffs remove routing boilerplate; built-in guardrails + tracing are production-friendly; little hidden abstraction. **Weaknesses:** OpenAI-centric defaults; fewer heavy orchestration primitives than LangGraph (no rich graph/checkpoint model); newer, smaller ecosystem.

6. Anthropic Claude Agent SDK — the harness behind Claude Code

What it is. Anthropic’s official SDK exposing the **same agent loop, tools, and context-management infrastructure that powers Claude Code**. Ships in **Python and TypeScript**.

Naming/history. Stable **1.0 on Sept 29, 2025**; renamed from `claude-code-sdk` → `claude-agent-sdk` (Python) and `@anthropic-ai/claude-code` → `@anthropic-ai/claude-agent-sdk` (TS/JS). The Claude Code CLI is bundled and driven under the hood.

Primitives (verified):

- **Agent-loop control** — interactive/bidirectional conversation via `ClaudeSDKClient`, or one-shot `query()`.
- **Built-in tools** — Claude Code’s full toolset (Read, Write, Edit, Bash, ...).
- **In-process MCP servers** — define custom tools as Python/TS functions running *inside* your app (`create_sdk_mcp_server`), no separate subprocess; mix with external MCP servers.
- **Hooks** — functions invoked at specific points in the loop for deterministic processing, automated feedback, or blocking dangerous commands.
- **Subagents** — delegate to specialized child agents.
- **Permission modes** — fine-grained tool-access gating.
- **Context management & sessions** — working dir, system prompts, config.

Use it when	Avoid / reconsider when
You're building coding / computer-use / file-operating agents on Claude and want the proven Claude Code loop + in-process MCP tools, hooks, subagents, permission gating.	You're not on Claude, or you want a vendor-neutral general orchestration toolkit.

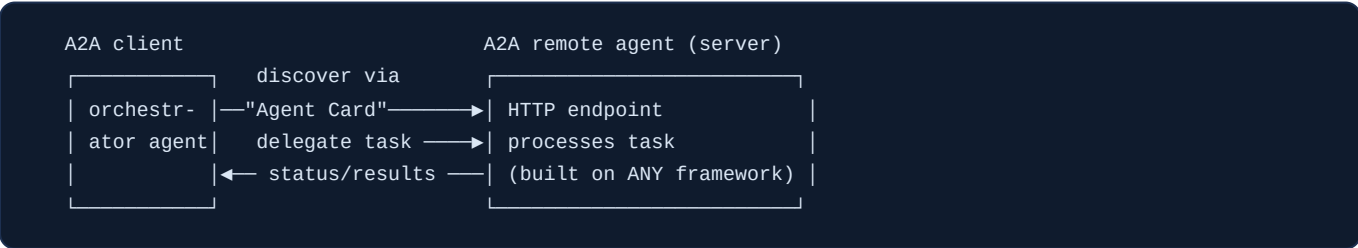
Strengths: inherits a battle-tested coding-agent harness (context compaction, tool orchestration); strong on filesystem/coding/computer-use; first-class MCP, hooks (deterministic safety), and subagents; runs inside your own program. **Weaknesses:** Claude/Anthropic-centric; the bundled Claude Code CLI dependency + harness assumptions are a heavier substrate than a thin SDK; more an *agent-runtime* than a general graph toolkit.

7. Google Agent Development Kit (ADK) + A2A

ADK. Open-source agent framework on the **same stack that powers Google Agentspace / Customer Engagement Suite** agents. Multi-language: **Python, Java (1.0), and Go** (Go added A2A support). Integrates with Vertex AI Agent Engine for managed deployment.

A2A (Agent2Agent) — the interop story. Open standard introduced by Google **April 2025** for cross-agent, cross-vendor, **cross-framework** communication (agents on ADK, LangGraph, CrewAI, etc. can interoperate).

- **Roles:** A2A **client** (delegates tasks) ↔ A2A **server / remote agent** (exposes an HTTP endpoint, processes tasks, returns status/results).
- **Agent Cards:** machine-readable descriptors (name, description, version, endpoint URL, transports, auth requirements) for discovery.
- **v0.3 upgrade:** more stable interface, **gRPC support, signed security cards**, extended Python SDK client support — aimed at enterprise adoption.



Use it when	Avoid / reconsider when
You need cross-vendor / cross-framework interoperability (A2A) for multi-agent systems, especially on Google Cloud / Vertex AI .	Single-framework, single-vendor app with no interop need.

Strengths: the open interop protocol (A2A) is the differentiator — escape single-vendor lock-in; enterprise Google Cloud + Vertex integration; multi-language. **Weaknesses:** newer; large protocol-and-framework surface area; deepest value tied to the Google Cloud ecosystem.

8. Microsoft Semantic Kernel — enterprise .NET-first SDK

Model. Code-first orchestration SDK around a central **Kernel** object that runs the model, registers/invokes **plugins**, and (historically) planners. Language-agnostic (**C#, Python, Java**) and model-agnostic (Azure OpenAI, OpenAI, others), with strong DI integration in .NET.

Primitives. Plugins (skills/functions — the tool-calling unit), **function calling** (now the recommended path), memory/connectors, and an **Agent Framework** layer for single- and multi-agent orchestration.

2025 shift (important): Planners (`HandlebarsPlanner` , `FunctionCallingStepwisePlanner`) are deprecated for new agents — use **function calling** instead for better flexibility, tool support, and simpler DX. And per §4, SK is **merging with AutoGen into the unified Microsoft Agent Framework**.

Use it when	Avoid / reconsider when
You're a .NET / Microsoft-stack enterprise wanting typed, DI-friendly plugins + function calling with Azure integration and enterprise security.	You want autonomy-first agent framing, or you're outside the MS ecosystem.

Strengths: best fit for .NET/enterprise teams; typed tooling, DI patterns, Azure integration, mature production posture. **Weaknesses:** planner deprecation + Agent Framework consolidation = ecosystem in transition; less “agentic autonomy” framing than CrewAI/AutoGen; heavier enterprise patterns than lightweight SDKs.

9. LlamaIndex Agents — data/RAG-rooted

Model. AgentWorkflow orchestrates one or more agents, built on LlamaIndex’s lower-level **Workflow** (event-driven, step-based) abstraction; it handles coordination, state, and hand-off between agents.

Agent classes. `FunctionAgent` (LLM function/tool calling) and `ReActAgent` (ReAct loop for non-function-calling models) — both inherit from `BaseWorkflowAgent` . AgentWorkflow picks the right one based on model capability.

Primitives. Workflow steps/events, the AgentWorkflow orchestrator, FunctionAgent/ReActAgent, tools, shared state; single specialized agent or collaborative multi-agent teams.

Use it when	Avoid / reconsider when
Your agent is data/RAG-centric — reasoning over large document/knowledge corpora is the core, orchestration layered on top.	General multi-agent control with no retrieval focus (LangGraph is stronger there).

Strengths: deep roots in RAG / data ingestion / retrieval — strongest when agents reason over large corpora; flexible Workflow primitive underneath; stateful multi-agent support. **Weaknesses:** orchestration is newer/lighter than LangGraph; primary gravity is data/retrieval, not general control.

10. Master decision table — “pick X when...”

Framework	Pick it when...	Avoid / reconsider when...
None (direct API + the 5 patterns)	The task fits a single call or one workflow pattern; you want max debuggability; you're prototyping.	You genuinely need durable state, many specialized agents, or org-wide standardization.
LangGraph	Explicit, auditable control over complex flows with loops, retries, branching, durable state, multi-actor orchestration.	Quick prototype / simple linear flow; team can't absorb graph + state-schema modeling.
CrewAI (Crews)	Work decomposes into human-team-like roles that collaborate autonomously.	You need strict, predictable control paths (use Flows or LangGraph).
CrewAI (Flows)	Event-driven, deterministic orchestration that can also embed autonomous crews.	A single autonomous crew already suffices.
AutoGen / AG2	Conversational multi-agent / code-execution loops; comfortable tracking the ecosystem.	You need stability now — watch the Microsoft Agent Framework consolidation.
OpenAI Agents SDK	On/near OpenAI models; want a thin layer with handoffs, guardrails, sessions, tracing.	You need a rich graph engine or heavy non-OpenAI orchestration.
Claude Agent SDK	Coding / computer-use / file-operating agents on Claude; want the Claude Code loop + MCP tools, hooks, subagents, permission gating.	Not on Claude, or you want a vendor-neutral general orchestration toolkit.
Google ADK + A2A	Cross-vendor / cross-framework interoperability, especially on Google Cloud / Vertex AI.	Single-framework single-vendor app with no interop need.
Semantic Kernel	.NET / Microsoft-stack enterprise; typed plugins + function calling with Azure integration.	Autonomy-first framing, or outside the MS ecosystem.
LlamaIndex (AgentWorkflow)	Data/RAG-centric agent over large corpora.	General multi-agent control with no retrieval focus.

11. Cross-cutting protocols: A2A vs MCP

A frequent point of confusion — they solve **different** problems and are designed to be used *together*:

	MCP (Model Context Protocol)	A2A (Agent2Agent)
Connects	An agent ↔ tools and data	An agent ↔ other agents
Question it answers	“What can this agent <i>do/read</i> ?”	“How do agents <i>delegate to each other</i> ?”
Heavily used by	Claude Agent SDK; supported broadly	Google ADK; cross-framework
2025 state	Widely adopted tool standard	Google-led; v0.3 adds gRPC + signed cards

They are standards, not frameworks — and increasingly the interop glue between any of the frameworks above. An A2A “remote agent” can itself be built on LangGraph or CrewAI and expose MCP tools internally.

12. Choosing — a short algorithm

1. **Can a single augmented LLM call or one of the 5 workflow patterns do it?** → Use direct API calls. Stop here for most tasks.
2. **Do you need durable, resumable, looping/branching control you can audit?** → LangGraph.
3. **Is the work naturally a collaborating team of roles?** → CrewAI Crews (+ Flows for the deterministic spine).
4. **Are you committed to a vendor’s models/runtime?** → OpenAI Agents SDK (OpenAI), Claude Agent SDK (Claude, especially coding/computer-use), Semantic Kernel (.NET/Azure), Google ADK (Vertex).
5. **Is retrieval over large corpora the core?** → LlamaIndex AgentWorkflow.
6. **Must agents from different teams/vendors interoperate?** → adopt **A2A** as the seam; use **MCP** for tools regardless of framework.
7. **Whatever you pick, understand the code underneath** and be ready to drop abstraction in production.

Sources

Primary / authoritative

- Building Effective AI Agents — Anthropic — <https://www.anthropic.com/research/building-effective-agents>
- OpenAI Agents SDK (official docs) — <https://openai.github.io/openai-agents-python/>
- Guardrails — OpenAI Agents SDK — <https://openai.github.io/openai-agents-python/guardrails/>
- Agents SDK — OpenAI API guide — <https://developers.openai.com/api/docs/guides/agents>
- claude-agent-sdk-python (GitHub, Anthropic) — <https://github.com/anthropics/claude-agent-sdk-python>
- AutoGen to Microsoft Agent Framework Migration Guide — <https://learn.microsoft.com/en-us/agent-framework/migration-guide/from-autogen/>
- Microsoft Agent Framework Overview — <https://learn.microsoft.com/en-us/agent-framework/overview/>
- AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation — Microsoft Research — <https://www.microsoft.com/en-us/research/publication/autogen-enabling-next-gen-llm-applications-via-multi-agent-conversation-framework/>
- Multi-agent Conversation Framework | AutoGen 0.2 — https://microsoft.github.io/autogen/0.2/docs/Use-Cases/agent_chat/
- Semantic Kernel (GitHub, Microsoft) — <https://github.com/microsoft/semantic-kernel>

- What are Planners in Semantic Kernel — Microsoft Learn — <https://learn.microsoft.com/en-us/semantic-kernel/concepts/planning>
- Configuring Agents with Semantic Kernel Plugins — Microsoft Learn — <https://learn.microsoft.com/en-us/semantic-kernel/frameworks/agent/agent-functions>
- ADK with Agent2Agent (A2A) Protocol — Google ADK docs — <https://google.github.io/adk-docs/a2a/>
- Introduction to A2A — Google ADK docs — <https://google.github.io/adk-docs/a2a/intro/>
- Agent2Agent protocol is getting an upgrade (v0.3) — Google Cloud Blog — <https://cloud.google.com/blog/products/ai-machine-learning/agent2agent-protocol-is-getting-an-upgrade>
- What's new with Agents: ADK, Agent Engine, and A2A Enhancements — Google Developers Blog — <https://developers.googleblog.com/agents-adk-agent-engine-a2a-enhancements-google-io/>
- Announcing ADK for Java 1.0.0 — Google Developers Blog — <https://developers.googleblog.com/announcing-adk-for-java-100-building-the-future-of-ai-agents-in-java/>
- Build and manage multi-system agents with Vertex AI — Google Cloud Blog — <https://cloud.google.com/blog/products/ai-machine-learning/build-and-manage-multi-system-agents-with-vertex-ai>
- CrewAI (GitHub) — <https://github.com/crewaiinc/crewai>
- CrewAI Flows — <https://crewai.com/crewai-flows>
- LlamaIndex AgentWorkflow blog — <https://www.llamaindex.ai/blog/introducing-agentworkflow-a-powerful-system-for-building-ai-agent-systems>
- LlamaIndex Agent Classes (API ref) — https://docs.llamaindex.ai/en/stable/api_reference/agent/
- LlamaIndex Agents (module guide) — https://docs.llamaindex.ai/en/stable/module_guides/deploying/agents/

Secondary / corroborating

- Claude Agent SDK: Subagents, Sessions and Why It's Worth It — <https://www.ksred.com/the-claude-agent-sdk-what-it-is-and-why-its-worth-understanding/>
- Claude Agent SDK Complete Guide — https://hidekazu-konishi.com/entry/claude_agent_sdk_complete_guide.html
- OpenAI Agents SDK vs Swarm: Migration Guide — <https://www.respan.ai/articles/openai-agents-sdk-vs-swarm>
- The AI Agent Framework Landscape in 2025 (Medium) — <https://medium.com/@hieutrantrung.it/the-ai-agent-framework-landscape-in-2025-what-changed-and-what-matters-3cd9b07ef2c3>
- The Developer's Guide to AI Agent Frameworks in 2025 (DEV) — https://dev.to/hani__8725b7a/agent-ai-frameworks-comparison-2025-mcp-agent-langgraph-ag2-pydanticai-crewai-h40
- Best Multi-Agent Frameworks in 2026 (gurusup) — <https://gurusup.com/blog/best-multi-agent-frameworks-2026>

Context Engineering & Memory for Agents

Source: [readme/research/agent-orchestration/AGENT_ORCHESTRATION_05_CONTEXT_MEMORY_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is about how AI agents manage the information they hold in mind while working. Every AI has a limit on how much it can pay attention to at once — and jamming more in does not help, it actually makes performance worse. This doc explains the smart ways to keep agents fast, cheap, and accurate despite that limit.

The main parts explained simply:

- **The attention budget** — An AI can only focus on a certain amount of text at once. More text does not mean better answers; past a point it gets confused, misses details, and costs more money. Think of it like a person trying to read 50 pages of notes at once vs a clean one-page summary.
- **Context rot / “lost in the middle”** — Research on 18 AI models found that every single one got worse as more text was loaded in. Facts buried in the middle of a long document were the most likely to be missed. Important instructions should go at the very start or the very end, never buried in the middle.
- **Context engineering** — The skill of choosing *what* to put in front of the AI, not just *how* to phrase it. It covers the whole package: instructions, examples, tool descriptions, history, retrieved data — everything the AI sees.
- **Compaction / summarization** — When a long task is running and the AI’s “notepad” fills up, the system summarizes the old work into a short digest and discards the raw details. This frees up space without losing the key decisions made so far.
- **Memory tiers** — Short-term memory lasts only for one conversation. Long-term memory persists across sessions and stores facts about users, past tasks, and rules the agent should always follow. Using both together lets agents feel consistent and “remember” returning customers.
- **Saving to files (offloading)** — Instead of keeping everything in mind, smart agents write notes to files and read them back only when needed. This is like keeping a running to-do list rather than trying to memorize everything.
- **Just-in-time retrieval** — Rather than loading a full database up front, the agent looks up only what it needs, right when it needs it. Faster and cheaper than pre-loading everything.
- **Sub-agents with separate workspaces** — For big research tasks, you can run several smaller agents in parallel, each focused on one slice of the problem, then combine their summaries. Each agent gets a clean, focused workspace. For tasks where agents must share and write to the same data, a single agent works better.
- **Prompt caching** — If the opening part of every AI request is identical (the instructions, the tool list), the system can reuse the computed result instead of processing it again. This cuts costs by up to 90% and speeds up responses significantly.

What to do with this: Keep AI instructions short, structured, and placed at the start. Use file-based notes for long tasks so the agent doesn’t lose track of earlier steps. Enable prompt caching for any repeated workflow — it is the single cheapest performance win available.

How to spend a finite attention budget: curate the smallest high-signal token set, offload the rest, and cache the stable prefix.

Research/reference doc · 2026-06-16 · part of the Agent Orchestration suite

0. The one idea this doc is built on

More context is not free, neutral, or harmless. It costs **money**, adds **latency**, and — past a point — **degrades accuracy**. The discipline of context engineering is finding “the smallest set of high-signal tokens that maximize the likelihood of your desired outcome” (Anthropic).

Four recurring tactics organize everything below. Manus names these dimensions explicitly; they map cleanly onto the rest of this doc:

Tactic	What it does	Sections
Reduce	Shrink what’s in the window (compaction, tool-result clearing, summarization)	§4
Offload	Move state out of the window (files, memory tool, vector store, scratchpads)	§5, §6, §9
Retrieve just-in-time	Keep identifiers, load data at runtime via tools	§7
Isolate	Give subtasks their own clean window	§8

A fifth lever — **caching** — is orthogonal: it doesn’t reduce the tokens the model sees, but it slashes the cost/latency of the *stable prefix*, which rewards append-only, byte-stable context design (§10).

1. The context window as a scarce budget

Anthropic frames context as a “finite resource with diminishing marginal returns.” Like a human’s limited working memory, an LLM has an **attention budget** that depletes with every token added.

Why it’s mechanical, not a soft preference. Transformer attention computes over **n^2 pairwise relationships** between tokens. As the sequence grows, a fixed attention capacity is spread across more relationships, so focus on any individual token weakens. The governing tension: *more context length ⇔ less attention focus*.

The stakes, in tokens. Token usage is the dominant lever on both cost and quality:

- Agents use **~4× more tokens** than a chat interaction.
- Multi-agent systems use **~15× more tokens** than a chat.
- In Anthropic’s BrowseComp evaluation, **token usage alone explained 80% of the performance variance** (tool calls and model choice explained the rest).

The practical restatement, applied to every component of the payload (system prompt, tools, examples, history): keep it “**informative, yet tight.**”

2. Context rot / lost-in-the-middle

Context rot (Anthropic): “As the number of tokens in the context window increases, the model’s ability to accurately recall information from that context decreases.” This is measured degradation, not just hitting a hard limit.

The empirical backbone — Chroma “Context Rot” (2025)

Chroma tested **18 frontier LLMs** across four labs (Claude, OpenAI, Gemini, Qwen) on extended Needle-in-a-Haystack variants, LongMemEval (~113k-token conversational Q&A), and a “Repeated Words” task. Headline: “Models do not use their context uniformly; instead, their performance grows increasingly unreliable as input length grows.” **Every model degraded at every input-length increment tested.**

Finding	Detail
Needle–question similarity matters	Lower semantic similarity between the question and the target fact degrades faster as context grows.
Distractors compound	“Even a single distractor reduces performance relative to the baseline (needle only), and adding four distractors compounds this degradation further.”
Haystack structure (counterintuitive)	Models do <i>worse</i> when surrounding text has coherent logical flow; shuffling the haystack (removing local coherence) improved performance.
Position effects	In Repeated Words, accuracy is highest when the unique token sits near the beginning — classic lost-in-the-middle.
LongMemEval gap	Large accuracy drop between focused prompts (~300 tokens) and full prompts (~113k tokens), across all model families.

Central conclusion: “What matters more is how that information is presented” than its mere presence.

The foundational paper — “Lost in the Middle” (Liu et al., 2023; TACL 2024)

Varying the **position** of the relevant fact in multi-document QA and key-value retrieval produces a **U-shaped curve**: accuracy is highest when the relevant info is at the **beginning or end**, and degrades significantly (often **>30%**) when it’s in the **middle** — “even for explicitly long-context models.”

Later (2025) refinements: the U-shape holds mainly when the context is **< 50% full**; above ~50% full, degradation is by **distance from the end** (recency bias: recent > middle > early). Degradation also shows up on broader, more complex task types with **far fewer tokens** than needle tasks need.

Mechanism, in one line: quadratic attention dilution — as softmax attention spreads over a long sequence, signal-to-noise drops and salient mid-context tokens get drowned out.

Design implications: put the most load-bearing instructions at the **start or end** of the window; minimize distractors; don't assume a long-context model "just handles it."

3. Context engineering vs prompt engineering

	Prompt engineering	Context engineering
Definition	<i>"Writing and organizing LLM instructions for optimal outcomes"</i>	<i>"Strategies for curating and maintaining the optimal set of tokens (information) during LLM inference, including all the other information that may land there outside of the prompts"</i>
Scope	Words/phrasing of one prompt	The whole inference-time payload: system prompt, tools, examples, history, retrieved data, tool outputs, memory
Cadence	One-time authoring act	Iterative curation , <i>"each time we decide what to pass to the model"</i>

Context engineering is *"the natural progression of prompt engineering."*

The "right altitude" principle (system-prompt design)

Aim between two failure modes:

1. **Too specific** — *"hardcoding complex, brittle logic"* → fragile.
2. **Too vague** — *"high-level guidance that fails to give the LLM concrete signals."*

Best prompts are *"specific enough to guide behavior effectively, yet flexible enough to provide the model with strong heuristics."* Structure with **XML tags or Markdown headers**; target the *"minimal set of information that fully outlines your expected behavior."*

Method: start with the best model + a minimal prompt, then add instructions/examples to patch observed failure modes — not speculatively.

Tools and examples

- **Tools** should be self-contained, token-efficient, and minimally overlapping. *"If a human engineer can't definitively say which tool should be used in a given situation, an AI agent can't be expected to do better."* Avoid bloated tool sets with ambiguous decision points.
- **Few-shot examples:** don't stuff edge cases. Curate *"a set of diverse, canonical examples that effectively portray the expected behavior"* — for an LLM, *"examples are the 'pictures' worth a thousand words."* (But beware mimicry — see §9.)

4. Compaction & summarization (Reduce)

Compaction (Anthropic): when a conversation nears the context limit, **summarize and reinitialize** with a compressed summary. Pass the message history back to the model to compress; *"the model preserves*

architectural decisions, unresolved bugs, and implementation details while discarding redundant tool outputs or messages.”

Tuning method: maximize **recall first** (capture every relevant detail), then iterate on **precision** (cut superfluous content).

Cheapest variant — tool-result clearing: drop raw tool outputs once their results are recorded. The lightest form of compaction; the cleared content is *removed*, not summarized.

Productized on the Claude Developer Platform (Sonnet 4.5+ launch; public beta, also on Amazon Bedrock and Google Vertex AI)

Feature	What it does
Context editing	Automatically clears stale tool calls/results as the model approaches the token limit, keeping recent ones and preserving relevant state. (Prunes — distinct from compaction’s summarize.)
Memory tool (file-based)	Stores/retrieves knowledge outside the window, persisting across sessions.

Reported internal results:

- Context editing alone: **+29%** on agentic search.
- Context editing + memory tool: **+39%**, plus an **84% reduction in token consumption** across 100-turn evaluations.

On the API, server-side compaction is a beta (`compact-2026-01-12`) on Fable 5 / Opus 4.8 / 4.7 / 4.6 / Sonnet 4.6: the API summarizes earlier context as it nears a trigger threshold (default ~150K tokens) and returns a **compaction block** you must append back (the full `response.content` , not just the text) — extracting only the text silently loses the compaction state.

Cognition’s note: for truly long tasks, introduce a dedicated LLM whose sole job is to **compress action/conversation history into key details, events, and decisions**.

5. Memory tiers

Short-term (working) vs long-term (persistent) — LangChain/LangGraph

Aspect	Short-term (working)	Long-term (persistent)
Scope	Thread-scoped — one conversation/session	Across sessions , shared across threads
Contents	Message history + stateful data (uploaded files, retrieved docs, generated artifacts)	User-/app-level facts, past experiences, learned rules
Storage	Agent state , persisted via a checkpoint er so a thread can resume	JSON docs in a store (<code>BaseStore</code>), under a namespace (folder) + key (filename); supports semantic search + filtering
Management	Trimming / summarization (full history may not fit)	Recalled on demand in any thread

Long-term subtypes (the human analogy: facts / experiences / rules)

Type	Definition	Agent use	LangGraph realization
Semantic	Specific facts and concepts	Personalization; user facts & preferences	Stored facts/profile docs
Episodic	Recalling past events/actions (with metadata: timestamps, participants)	Remember <i>how</i> a task was accomplished via concrete examples	Stored interaction examples / few-shot history
Procedural	The rules / how-to (generalized skills & behaviors)	Govern the agent's behavior	Often the system prompt/instructions , self-updated over time

When memories are written

- **Hot path** (during runtime): immediately usable, but adds complexity and **latency**.
- **Background** (“subconscious”): a separate process forms memories — **no latency** in the main loop, but you must decide **when to trigger** formation.

(LangMem layers these three types on top of LangGraph’s `BaseStore` .)

Connecting the vocabularies: long-term memory is the *store*; context engineering decides *what to page into the window* and *when*. The store is large and persistent; the window is small and budgeted.

6. External memory stores (Offload)

Structured note-taking / agentic memory (Anthropic): agents write persistent notes **outside the context window** and pull them back later. This lets agents “*track progress across complex tasks, maintaining*

critical context and dependencies that would otherwise be lost across dozens of tool calls.”

- Example — Claude playing Pokémon keeps precise tallies across thousands of steps: “for the last 1,234 steps I’ve been training my Pokémon in Route 1, Pikachu has gained 8 levels toward the target of 10.”
- `CLAUDE.md`-style files are an example of memory dropped naively into context up front.

Manus — “filesystem as the ultimate context”: treat the file system as memory that is “unlimited in size, persistent by nature, and directly operable by the agent.” Critical rule — **compression must be restorable**:

Drop the webpage *content* but keep the **URL**; omit a document’s *text* but keep its **file path**. Nothing is irreversibly lost while in-context tokens shrink.

Three external-memory substrates seen across sources:

Substrate	Recall mode	Examples
Vector DBs	Semantic recall	LangGraph <code>BaseStore</code> (JSON + namespaces + semantic search)
Files / scratchpads	Path/name lookup	Anthropic memory tool, Manus filesystem, <code>todo.md</code>
Structured state objects	Checkpointed	LangGraph thread state via checkpointer

7. RAG vs agentic search / just-in-time retrieval

	Pre-retrieval (traditional RAG)	Just-in-time (agentic search)
When	Embed + fetch chunks up front , before inference	Agent loads data at runtime via tools as it reasons
What’s in context	The fetched chunks	Lightweight identifiers (file paths, stored queries, web links)
Speed	Faster (pre-computed)	Slower (runtime exploration)
Exemplar	Classic RAG pipeline	Claude Code — targeted queries, <code>head</code> / <code>tail</code> / <code>grep</code> over large data <i>without loading full objects</i>

Metadata is signal. “Folder hierarchies, naming conventions, and timestamps all provide important signals” guiding what to load and when.

The trade-off and the winner. Runtime exploration is slower than pre-computed retrieval; a **hybrid** (some upfront retrieval + autonomous exploration) often wins.

Where the field is going (2025 agentic-RAG survey). From **static, rule-based** pipelines toward **dynamic, reasoning-driven** ones that embed retrieval *decisions into the reasoning loop* — the model decides **when, what, and how** to retrieve (ReAct, Self-Ask, Search-o1 interleave generation with retrieval;

RL-trained tool-invocation policies). Traditional RAG struggles with multi-step investigative queries that need reasoning, not just fact lookup. **Open problem:** agentic retrieval can over-call tools (redundant retrievals); the research direction is **adaptive retrieval** that fires only when the model's internal knowledge is insufficient.

8. Sub-agent context isolation (Isolate)

Anthropic's multi-agent research system (orchestrator-worker): a **lead agent** plans, **spawns 3–5 specialized subagents in parallel**, then synthesizes (with a separate citation pass).

Each subagent has its own context window. *“Subagents facilitate compression by operating in parallel with their own context windows, exploring different aspects of the question simultaneously before condensing the most important tokens for the lead research agent.”*

- **Separation of concerns** — distinct tools, prompts, trajectories — *“reduces path dependency and enables thorough, independent investigations.”*
- The **detailed search context stays isolated inside subagents**; only a **condensed, distilled summary (often 1,000–2,000 tokens)** returns to the lead.
- **Why it works:** the system can reason over **more aggregate context than a single window can hold**; ideal for **breadth-first** queries needing many independent paths (e.g., “identify all board members of IT companies in the S&P 500”).

Performance & economics. Multi-agent (Opus 4 lead + Sonnet 4 subagents) **outperformed single-agent Opus 4 by 90.2%** on the internal research eval — at **~15× the tokens of a chat**. Only worth it for **high-value, heavily parallelizable** tasks where info exceeds a single window and tools are numerous/complex. The lead **saves its plan to Memory** because *“if the context window exceeds 200,000 tokens it will be truncated”* — it retrieves the plan rather than losing work.

Counterpoint — Cognition, “Don’t Build Multi-Agents”

Multi-agent setups are **fragile**: subagents lack context of each other's work, decisions get dispersed, and failures *“boil down to missing context.”*

- Recommends **single-threaded, linear agents** with **continuous context**.
- The **“single-writer” principle**: keep **writes/actions single-threaded**; extra agents contribute **intelligence (read-only analysis), not actions**.
- For long tasks, use a dedicated **history-compression model** rather than parallel writers.

WHEN ISOLATION HELPS vs HURTS	
Independent + read-mostly (breadth-first research, parallel investigations)	→ isolate (Anthropic) clean windows win
Shared state / coordinated writes / dependencies (refactors, write-heavy pipelines)	→ single-thread (Cognition) continuous context wins

Reconciliation: isolation helps when subtasks are **independent and read-mostly**; it hurts when they must **share state or coordinate writes**.

9. Structured note-taking / state files

Beyond §6's "offload to survive dozens of tool calls," Manus surfaces three sharper tactics:

Recitation to steer attention. Maintain a `todo.md`, continuously rewritten with items checked off. This "*recites objectives into the end of the context*" — pushing goals into the model's **most recent attention span** and countering drift / lost-in-the-middle over long runs (Manus averages **~50 tool calls** per task).

⚠ Caveat: an early naive version spent **~one-third of all actions** just updating the todo list. Structure note-taking to be cheap, and consider a dedicated planner.

Keep errors in context. Don't hide failed actions, retries, or stack traces. Seeing a failure lets the model "*implicitly update internal beliefs*," shifting priors away from repeating the mistake — error recovery is "*one of the clearest indicators of true agentic behavior*."

Avoid few-shot mimicry. LLMs are "*excellent mimics*"; uniform repeated context makes agents fall into a brittle "rhythm." Inject **small structured variation** (alternate templates/phrasing/ordering) to break harmful pattern-copying. (This is the tension with §3's "curate canonical examples" — diversity matters as much as quality.)

10. Prompt caching to cut cost on stable prefixes

Caching doesn't reduce tokens the model processes — it reuses the **compute** for an unchanged prefix, slashing cost and latency.

The one invariant

Prompt caching is a prefix match. Any byte change anywhere in the prefix invalidates everything after it. Render order is fixed: `tools` → `system` → `messages`. Keep stable content first; put volatile

content (timestamps, per-request IDs, the incoming question) **after the last `cache_control` breakpoint.**

[tools]

[system]

[messages]

← most stable

← stable

← grows per turn

└

|

┘

put breakpoint on the LAST block

whose prefix is identical across requests

never on volatile content

Mechanics (authoritative facts)

Knob	Value
Breakpoints per request	Max 4 (<code>cache_control: {type: "ephemeral"}</code>)
Auto-caching	Top-level <code>cache_control</code> on <code>messages.create()</code> auto-places on the last cacheable block
Default lifetime	5-minute ephemeral cache (refreshed free on each use)
Optional lifetime	1-hour TTL (<code>"ttl": "1h"</code>), at higher write cost
Minimum cacheable prefix	Model-dependent — Opus 4.8/4.7/4.6/4.5 & Haiku 4.5: 4096 tokens ; Fable 5 / Sonnet 4.6 / Haiku 3.5: 2048 ; Sonnet 4.5/4/3.7: 1024 . Shorter prefixes silently won't cache.
Lookback	Each breakpoint walks back at most 20 content blocks to find a prior entry

Pricing (multipliers on base input price)

Operation	Multiplier
Cache read (hit)	0.1× base input (~90% cheaper than fresh input)
5-min cache write	1.25× base input
1-hour cache write	2× base input
Output tokens	unaffected

Break-even: with 5-min TTL, two requests break even ($1.25\times + 0.1\times = 1.35\times$ vs $2\times$ uncached); with 1-hour TTL you need ≥ 3 requests ($2\times + 0.2\times = 2.2\times$ vs $3\times$ uncached). Reported real-world savings: input-token costs cut **~70–90%**, latency cut **up to ~85%** for long prompts.

Manus corroboration — caching as a design constraint

KV-cache hit rate is “the single most important metric for a production-stage AI agent.” With Manus’s ~**100:1** input:output ratio, the prefix dominates cost; a cached vs uncached input is a **10×** difference. Cache-preserving practices (which align with everything else in this doc):

- Keep the **prompt prefix byte-stable** — a single changed token invalidates everything after it.
- **Avoid timestamps** in system prompts.
- Make context **append-only**.
- Use **deterministic JSON serialization** (stable key order).

- **Mask tools, don't add/remove them** — changing the tool list invalidates the cache *and* confuses the model. Use logit masking + consistent tool-name prefixes (`browser_` , `shell_`).

Verifying hits

Check the response `usage` : `cache_creation_input_tokens` (written this request), `cache_read_input_tokens` (served from cache), `input_tokens` (uncached remainder, *after the last breakpoint*). Total prompt size = the sum of all three. If `cache_read_input_tokens` is **zero** across repeated identical-prefix requests, a silent invalidator is at work (`datetime.now()` in the system prompt, unsorted JSON, a varying tool set) — diff the rendered prompt bytes between two requests.

11. Putting it together — a decision checklist

1. **Is the context near the window limit?** → **Reduce**: tool-result clearing first (cheapest), then compaction (maximize recall, then precision).
2. **Does state need to outlive the window or the session?** → **Offload** to files/memory tool/vector store. Make compression **restorable** (keep the path/URL).
3. **Is the data large but only partially needed?** → **Retrieve just-in-time** over identifiers (agentic search) rather than stuffing it up front.
4. **Are subtasks independent and read-mostly?** → **Isolate** in subagents with clean windows. If they share state or coordinate writes → keep a single writer (Cognition).
5. **Always, orthogonally**: design the prefix to be **append-only and byte-stable** so the cache stays warm — it's the cheapest 10× you'll get.

Underneath all five: spend the attention budget on the **smallest high-signal token set**, and put the load-bearing tokens at the **start or end** of the window where the model actually attends.

Sources

- Effective context engineering for AI agents — Anthropic — <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- How we built our multi-agent research system — Anthropic — <https://www.anthropic.com/engineering/multi-agent-research-system>
- Managing context on the Claude Developer Platform (context editing + memory tool) — Anthropic — <https://www.anthropic.com/news/context-management>
- Context Rot: How Increasing Input Tokens Impacts LLM Performance — Chroma — <https://www.trychroma.com/research/context-rot>
- Lost in the Middle: How Language Models Use Long Contexts (Liu et al.) — arXiv — <https://arxiv.org/abs/2307.03172>
- Lost in the Middle — ACL Anthology (TACL 2024) — <https://aclanthology.org/2024.tacl-1.9/>

- Memory overview — LangChain / LangGraph docs — <https://docs.langchain.com/oss/python/concepts/memory>
- LangMem SDK for agent long-term memory — LangChain blog — <https://www.langchain.com/blog/langmem-sdk-launch>
- Prompt caching — Claude API Docs — <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- Context Engineering for AI Agents: Lessons from Building Manus — <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Don't Build Multi-Agents — Cognition — <https://cognition.ai/blog/dont-build-multi-agents>
- Reasoning RAG via System 1 or System 2: A Survey on Reasoning Agentic RAG (2025) — arXiv — <https://arxiv.org/pdf/2506.10408>
- AI Agents, RAG, and Agentic Retrieval Techniques for Enterprise-Grade Search — [Kore.ai](https://www.kore.ai/blog/ai-agents-rag-agentic-retrieval) — <https://www.kore.ai/blog/ai-agents-rag-agentic-retrieval>

Reliability, Evaluation & Observability

Source: [readme/research/agent-orchestration/AGENT_ORCHESTRATION_06_RELIABILITY_EVAL_OBS_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document explains why AI agents that do long, multi-step jobs are hard to keep working reliably — and what you have to do to catch problems before they cause real damage. It matters because the more steps an AI agent takes, the more chances there are for small mistakes to grow into big failures that waste money and time.

The main parts explained simply:

- **Why agents fail more than simple AI calls** — A single question-and-answer AI is easy to check. But an agent that makes decisions, calls tools, and talks to other agents can fail in many of the same ways a team of workers can: one mistake early on quietly causes bigger mistakes later. The document shows the math: if each step works 95% of the time, a 20-step job only works out correctly about 36% of the time.
- **The 14 ways agents go wrong (MAST taxonomy)** — Researchers studied 1,600+ real agent runs and found 14 specific failure patterns grouped into three buckets: bad instructions/design (about 42% of failures), agents misunderstanding each other (about 37%), and agents that don't know when to stop or check their own work (about 21%). The key finding: most failures come from *how the system is designed*, not from the AI being “dumb” — so upgrading the model alone won't fix them.
- **Guardrails** — Safety fences you put around every action the agent takes: checking inputs before they go in, checking outputs before they go out, limiting how much the agent can spend, and requiring a human to approve anything risky. Think of it like putting spending limits and approval rules on a company credit card.
- **Evaluation (testing whether it actually works)** — You can't just check if the final answer is right, because two agents might take completely different paths and both be correct. The document explains three levels of testing: did the task succeed, was the path taken sensible, and which specific part broke? It also covers using another AI to judge the first AI's work — useful but not perfect on its own.
- **Observability and tracing** — Recording a detailed log of every step the agent took, like a flight data recorder. Without this, when something goes wrong you can't tell *where* it went wrong. There is an emerging industry standard (OpenTelemetry) for how these logs should be structured, though it is still changing as of 2026.
- **Testing challenges (non-determinism)** — Even with the exact same settings, an AI agent can give different answers each time it runs. This makes standard software testing very hard. The practical fix: run tests many times and compare averages, not single runs; use replay tests that re-use saved AI responses instead of calling the live AI every time.
- **Resumability and checkpointing** — Saving progress at each step so that if the agent crashes halfway through a long job, it can pick up from where it stopped instead of starting over from scratch. For expensive long jobs this is essential — restarting from zero wastes both money and time.
- **Real performance numbers** — A multi-agent setup (one lead AI directing several sub-agents) beat a single AI by over 90% on research tasks, and cut research time by up to 90%. The catch: it costs about 15 times more in AI tokens. This is why all the reliability work in this document matters — you are spending much more, so you need to be sure it is actually working.

What to do with this: Before running long AI agent jobs in production, put spending limits and human-approval checkpoints in place so mistakes cannot spiral out of control. Start measuring whether the agent's work is actually correct from day one — even a simple 20-case test set will show you problems early, before they cost you money.

How to keep long-running, multi-agent LLM systems from quietly going off the rails — and how to know when they do.

Research/reference doc · 2026-06-16 · part of the Agent Orchestration suite

1. Why agents break the old reliability model

A single prompt → response call is easy to reason about: one input, one output, fail or succeed. The moment an agent starts to **plan, call tools, hold state, and talk to other agents**, you inherit every classic distributed-systems failure class — race conditions, partial failures, inconsistent state, cascading errors, deadlock — and then add a probabilistic engine on top of it.

The long-horizon cliff. Frontier models hit near-100% success on tasks a human could do in under ~4 minutes, but success falls **below 10%** on tasks needing more than ~4 hours of human-equivalent work. As you push agents into longer-horizon work, **time-compounding failure modes become the dominant source of cost and unreliability** — not raw model IQ.

The compounding-error math. If each step is independently 95% reliable, a 20-step trajectory is only $0.95^{20} \approx 36\%$ reliable. Reliability decays geometrically with horizon length:

steps:	1	5	10	20	40
@95%/step:	95%	77%	60%	36%	13%
@99%/step:	99%	95%	90%	82%	67%

Early mistakes rarely stay local — they “cascade into subsequent steps, distorting reasoning, compounding misjudgments, and ultimately derailing the entire trajectory.” Once a cascade starts it is hard to reverse, so **early detection and correction matter far more than post-hoc repair**.

Anthropic's framing: **“Minor system failures can be catastrophic for agents.”** Because agents are long-running and stateful, a small fault deep into a run wastes everything done before it.

Two design consequences flow from this whole section, and the rest of the doc elaborates them:

1. Cut horizon length and step count wherever you can (each step is a dice roll).
2. Detect-and-correct mid-run beats restart-from-scratch (see §7 resumability).

2. Failure modes

2a. MAST — the empirical taxonomy

The anchor reference is “**Why Do Multi-Agent LLM Systems Fail?**” (arXiv 2503.13657), the first empirically grounded multi-agent failure taxonomy. It was built from **1,600+ annotated execution traces across 7 popular MAS frameworks**, covering coding/math/general-agent tasks on GPT-4, Claude 3, Qwen 2.5, and CodeLlama, validated by 6 expert annotators with inter-annotator agreement **Cohen’s κ = 0.88**. It defines **14 failure modes in 3 categories**.

The central, uncomfortable finding: **many failures stem from organizational / system *design*, not raw model capability**. You can’t model-upgrade your way out of a bad orchestration design.

Category-level share of failures:

Category	Share
Specification & system design	≈ 41.8%
Inter-agent misalignment	≈ 36.9%
Task verification & termination	≈ 21% (remainder)

Cat	Failure mode	Definition
FC1 Specification / System Design	FM-1.1 Disobey task specification	Ignores stated constraints / requirements
	FM-1.2 Disobey role specification	Acts outside its assigned role boundaries
	FM-1.3 Step repetition	Needlessly redoes completed steps → delay/error
	FM-1.4 Loss of conversation history	Context truncated; reverts to an earlier state
	FM-1.5 Unaware of termination conditions	Doesn't know when to stop (→ loops)
FC2 Inter-Agent Misalignment	FM-2.1 Conversation reset	Unwarranted restart losing context/progress
	FM-2.2 Fail to ask for clarification	Proceeds on ambiguous/unclear input
	FM-2.3 Task derailment	Drifts toward irrelevant/unproductive actions
	FM-2.4 Information withholding	Fails to share insight peers needed
	FM-2.5 Ignored other agent's input	Disregards a collaborator's recommendation
	FM-2.6 Reasoning-action mismatch	Reasoning diverges from the executed action
FC3 Task Verification / Termination	FM-3.1 Premature termination	Ends before objectives are met
	FM-3.2 No/incomplete verification	Skips checking outcomes → errors propagate
	FM-3.3 Incorrect verification	Validation done, but inadequately/wrongly

2b. Cascading subagent failures

“A misaligned agent produces an output that the next agent accepts as authoritative, and a verification failure that should have caught the problem is never triggered.” Multi-agent systems **amplify** error across independent agents rather than averaging it out. Anthropic notes that asynchronous orchestration would add parallelism but introduces fresh hazards: **“challenges in result coordination, state consistency, and error propagation across the subagents.”**

2c. Infinite loops / non-termination

Maps to MAST FM-1.5 (no termination awareness) and FM-1.3 (step repetition). Agents reinterpret goals differently and ping-pong; missing termination conditions are a top specification-class defect. **Mitigation:** explicit max-steps / max-cost budgets, loop detection (repeated identical tool calls / states), and forced termination.

2d. Tool misuse (“functional hallucination”)

Misusing tools/APIs: wrong tool, invalid arguments, or an unsafe plan. The function-calling / ToolScan literature breaks it into three categories:

Code	Error	Example
IFN	Incorrect Function Name	Calls a function not in the available set (a hallucinated tool)
IAN	Incorrect Argument Name	Hallucinated parameter names
IAV	Incorrect Argument Value	Wrong values, omitted required args, malformed/invalid JSON

Security angle: an agent told never to read secrets can **hallucinate a `read_secret` capability** — and if such a function happens to exist in the environment, the call executes. Hallucinated tool *names* can become real **privilege escalation**. Mitigations: strict JSON-Schema / Pydantic tool schemas, validate args before execution, **fail closed** on invalid args, and write precise tool descriptions with examples and constraints.

2e. Hallucinated coordination

Agents fabricate that a handoff or communication occurred, or act on outputs another agent never produced. Overlaps FM-2.6 (reasoning-action mismatch) and FM-2.4/2.5 (communication breakdowns), plus “format mismatches between agents” and “context loss during handoffs.” Treat every inter-agent handoff as an untrusted boundary: validate the payload shape and provenance, don’t assume the upstream agent did what it claimed.

2f. Deadlock / synchronous bottlenecks

Anthropic’s lead agent runs subagents **synchronously**, blocking until each finishes. This “simplifies coordination, but creates bottlenecks”: the lead can’t steer subagents mid-flight, subagents can’t coordinate with each other, and **one slow subagent stalls the whole system**. This is the practical head-of-line-blocking shape of “deadlock” in current production multi-agent systems — usually not a true lock cycle, but a slow dependency freezing the pipeline.

3. Guardrails

Guardrails = technical controls, validation layers, and policy enforcement that constrain agent behavior within safety/compliance/operational boundaries — block harmful outputs, block adversarial inputs, and keep agents within authorized scope.

Layered model

INPUT VALIDATION	context filtering, access control, prompt-injection / jailbreak detection, relevance + blocklist + safety classify
ACTION GUARDRAILS (every tool call)	per tool call: • authorization check • schema validation on arguments • rate limit • cost meter / budget ceiling • kill switch
OUTPUT VALIDATION	JSON-schema compliance BEFORE anything downstream consumes the output
PERMISSION GATING	least privilege per tool; act only within authorized scope
HUMAN-IN-THE-LOOP	pause the run for approval when risk exceeds an automation threshold

- **Input validation** — block requests carrying sensitive data, detect prompt injection/jailbreaks, prevent connecting to unauthorized APIs; relevance/keyword/blocklist filters and safety classifiers.
- **Action-level guardrails** — wrap every tool call with authorization, argument schema validation, rate limit, cost meter, and a kill switch. Whitelist API calls; put **budget ceilings on LLM spend** so a misconfigured agent can't spiral.
- **Output validation** — for structured outputs feeding downstream APIs, enforce JSON-schema compliance and validate before anything consumes it.
- **Human-in-the-loop checkpoints** — inject human judgment when risk exceeds automation thresholds; require human approval for production changes and high-stakes actions.

Vendor guidance

- **OpenAI** (*A Practical Guide to Building Agents* + Agents SDK): guardrails are functions or agents enforcing jailbreak prevention, relevance, keyword filtering, blocklists, and safety classification — “critical at every stage, from input filtering and tool use to human-in-the-loop intervention.” Human review **pauses the run** so a person/policy approves or rejects a sensitive action; this is **especially important early in deployment** to surface failures and edge cases and bootstrap the eval cycle. OpenAI's Guardrails library ships an eval framework reporting **precision, recall, F1** against labeled test data.
- **LangGraph** has native **tool scoping** and **human-in-the-loop primitives** (interrupt / approve). Proxy layers (Invariant Gateway, agentgateway) centralize cross-agent policy enforcement.

PF360 house-rule tie-in (CLAUDE.md §0). If a guardrail or integration isn't configured, never show a broken/empty/silent-failure state. Hide or disable the control, explain in plain language what's missing,

and link to where to set it up — e.g. “No safety classifier configured. Set one up → Settings › Guardrails.”

4. Evaluation

4a. Why exact-match fails for agents

Agents don’t follow predetermined paths. Anthropic: **“Even with identical starting points, agents might take completely different valid paths to reach their goal.”** Step-by-step exact validation is therefore impossible. Concretely: if the gold trajectory calls `search_web` but the agent calls `search_documentation` and **both legitimately solve the task**, exact match wrongly labels a success as a failure. Free-form text outputs with multiple valid answers also resist string equality. You need fuzzy / semantic matching.

4b. Outcome evals vs trajectory evals

	Outcome / end-state eval	Trajectory eval
Judges	The final result — “did the task succeed?”	The whole execution path: tools chosen, intermediate reasoning, turns
Strength	Tolerates many valid paths; Anthropic leans here for research agents and judges outcomes rather than processes	Catches failures masked by a correct-looking answer (e.g. skipped identity verification before a refund)
Weakness	An agent “can call every tool correctly and still fail the task,” and a correct answer can hide broken reasoning / lucky-but-wrong tool calls	Brittle if matched too strictly; needs a reference trajectory

Trajectory matching modes:

- **Exact match** — tool calls + reasoning must match the reference exactly (strict, brittle).
- **In-order** — required tools called in correct sequence; intermediate detail may vary.
- **Any-order** — all required tools called, sequence ignored.

Three eval levels to run together:

```
end-to-end → did the task succeed? (outcome)
trajectory → was the path efficient & sound? (process)
component → which retriever/tool/subagent broke? (isolation)
```

Use **deterministic checks** for exact things (tool-name correctness, output format) and **LLM-as-judge** for anything depending on free-form output.

4c. LLM-as-judge

- **Anthropic’s implementation:** a single LLM call scoring output against a rubric on **factual accuracy, citation accuracy, completeness, source quality, tool efficiency** — producing a **0.0–1.0 score plus**

a pass/fail grade. It scaled to “hundreds of outputs” and “aligned well with human judgment.”

- **Caveats from the literature: no judge excels everywhere.** Mitigate with structured rubrics that constrain output, multiple judge passes with aggregation, calibration against human-labeled examples, and monitoring for **consistency drift** over time. Production studies of multi-turn transaction agents still find judges miss a meaningful fraction of failures (the “catching ~one in five” blind-spot framing).

Judges are necessary but not sufficient.

- **Agent-as-judge** is the emerging extension: use an agent to evaluate another agent’s full *trajectory*, not just its end result.

4d. Small, high-signal eval sets

Anthropic started with **~20 queries representing real usage**. Early development moved success from **30% → 80%** — an effect so large that a tiny set was enough to detect it. **Start evaluating immediately; don’t wait for a large suite**, and resist over-engineering eval infrastructure before you have any signal.

4e. Human evaluation remains essential

Automation misses subtle biases and edge cases. Anthropic’s concrete example: human testers caught that early agents “**consistently chose SEO-optimized content farms over authoritative but less highly-ranked sources**” — a quality bias no automated metric flagged. Wire a production feedback loop: failures → annotation queue → corrected trajectories → regression tests, so the same bug can’t reach users twice.

5. Observability & Tracing

5a. Why agent observability ≠ LLM monitoring

“Agent failures appear in **multi-step causal chains, not at the individual-call level**, and require **full-session trace capture** to detect.” Non-deterministic run-to-run behavior makes debugging hard; without tracing you can’t tell a bad search query from poor source selection from a tool failure. Anthropic uses **full production tracing** and monitors “**decision patterns and interaction structures**” while deliberately **not** reading individual conversation content — privacy-preserving root-cause analysis.

5b. OpenTelemetry GenAI semantic conventions (the emerging standard)

Driven by OTEL’s **GenAI SIG since April 2024**, these standardize attribute names/types/enums for LLM calls, agent steps, vector-DB queries, token usage, cost, and quality. **Status: most GenAI conventions are still experimental (not API-stable) as of early-mid 2026** — adopt them, but pin versions and expect churn.

Span tree:

```

invoke_agent          (top-level agent run)
├─ chat               (one per LLM call)
│   └─ execute_tool   (one per tool call)
├─ chat
└─ embeddings

```

Agent conventions extend/override the base GenAI span conventions and add concepts for tasks, actions, agents, teams, artifacts, and memory.

Key `gen_ai.*` attributes:

Attribute	Meaning
<code>gen_ai.operation.name</code>	<code>chat</code> , <code>invoke_agent</code> , <code>execute_tool</code> , <code>embeddings</code>
<code>gen_ai.request.model</code>	model requested (e.g. <code>gpt-4o</code>)
<code>gen_ai.usage.input_tokens</code> / <code>gen_ai.usage.output_tokens</code>	token counts
<code>gen_ai.response.finish_reasons</code>	why generation stopped (<code>stop</code> , <code>tool_calls</code> , ...)
<code>gen_ai.agent.id</code>	agent identifier
<code>gen_ai.tool.name</code>	tool invoked
<code>gen_ai.token.type</code>	metric dimension distinguishing input vs output
<code>gen_ai.system_instructions</code> / <code>gen_ai.input.messages</code> / <code>gen_ai.output.messages</code>	prompt/completion content (only when content capture is enabled)

Metrics: `gen_ai.client.operation.duration` (latency histogram), `gen_ai.client.token.usage` (token histogram).

Microsoft + Cisco Outshift introduced multi-agent semantic conventions on top of OTel + **W3C Trace Context**, standardizing telemetry for quality, performance, safety, cost, tool invocations, and inter-agent collaboration. Microsoft Foundry / Azure Monitor App Insights trace LangChain, LangGraph, OpenAI Agents SDK, and Microsoft Agent Framework. **Datadog** natively supports the OTel GenAI conventions.

5c. Tooling landscape

Tool	Strength	OSS / self-host	Notes
LangSmith	LangChain/LangGraph-native: node-by-node state diffs, full execution graphs, model+tool breakdowns, replay against new model versions	No (managed)	Least friction on LangChain/LangGraph; 5,000 traces/mo free
Langfuse	Operational telemetry + cost analytics; framework-agnostic via OTel; ingests any LLM SDK/agent framework	Yes (Postgres + ClickHouse)	Leading OSS observability path
Arize Phoenix	Enterprise RAG eval & monitoring; agent trace capture, LLM-as-judge eval, RAG metrics, dataset mgmt; OpenTelemetry-native	Yes	OSS eval + observability
W&B Weave	Experimentation & prompt research	Partial	Best for experiment tracking

Pricing gotcha worth a callout. Billing **per trace** penalizes agent complexity: a 50-step autonomous agent costs the same as one LLM call if billed per *span*, but **50× more** if billed per *trace*. Factor the billing unit into your architecture and vendor choice.

6. Testing & determinism challenges

LLMs are inherently probabilistic, and this defeats naive regression testing.

- **Even at temperature 0, output is not guaranteed identical run-to-run** — numerical-precision effects under greedy decoding still cause drift. OpenAI states its API can only be “**mostly deterministic**” regardless of temperature; the `seed` parameter improves but does **not guarantee** reproducibility, because of system updates and **load-balancing across different hardware** (Thinking Machines Lab traces the root cause to numerical nondeterminism in inference).
- The same agent config (prompt, tools, model, orchestration) can yield different outputs across invocations due to **temperature sampling, model weight/version updates, tool-latency variation, and context-window effects**. This makes **reproducing a specific failure extraordinarily difficult**.

Regression strategy for non-deterministic agents:

1. Run a **replay regression suite as a CI gate first** — “fast, cheap, and deterministic” (replay journaled tool/LLM results instead of re-calling live).
2. **Aggregate over many sessions before comparing A/B arms**. A single session can diverge *within* an arm; compare distributions, not point samples. Use the **same seed policy across both arms**.
3. Use **deterministic assertions** for exact things (tool-name correctness, schema/format) and **LLM-judge** for free-form quality.

Mantra: “**Temperature defines how wild the model can be; seeds make that wildness replayable**” — but seeds are a partial, not complete, solution.

7. Resumability, checkpointing & durable execution

The compounding-error math (§1) means **restarting a failed long run is wasteful**. The fix is to persist progress and resume from the failure point.

Anthropic’s production approach (multi-agent research system)

- **Don’t restart on failure** — it’s “expensive and frustrating.” **Resume from where the agent was when the errors occurred.**
- Combine **AI adaptability with deterministic safeguards: retry logic + regular checkpoints**, so agents recover gracefully when tools fail.
- **Context-window management for hundreds-of-turns runs:** agents summarize completed phases, offload essential info to **external memory**, then **spawn fresh subagents with clean contexts**, handing off to maintain continuity before hitting limits.
- **Rainbow deployments:** because the system runs almost continuously as a stateful web of prompts/tools, updates **gradually shift traffic old → new while running both simultaneously**, so in-flight agents aren’t disrupted.

Durable execution

Durable execution = save progress at key points so a process can pause and **resume exactly where it left off**; state is persisted after every logical step, so a crash resumes from the **last checkpoint, not the start**.

Framework	How it resumes	Watch out for
LangGraph	Persistence layer saves graph state as checkpoints organized into threads ; a <code>StateSnapshot</code> at every super-step; <code>thread_id</code> is the primary key. Interrupt pauses, saves, waits indefinitely (enables human-in-the-loop), then resumes.	Checkpointers save state between nodes, not inside a node; the developer must detect/trigger/coordinate resumption.
Temporal	Replays an Event History (durable log) to reconstruct in-memory state after a crash; resumes at the exact failure step without re-running completed work.	Workflow code must be deterministic . Because LLM calls are non-deterministic, wrap them as “activity” steps whose results are journaled on first execution and never re-run on replay — the key bridge between durable execution and probabilistic LLMs.
Restate / DBOS	Journals + stored step results. OpenAI’s Agents SDK docs point to DBOS for long-running agents, human-in-the-loop flows, and preserved progress across failures/restarts.	Same determinism discipline at the step boundary.

Caveat (Diagrid): plain checkpointing ≠ full durable execution. Checkpoints give you a save point but leave detection, coordination, and dedup to the developer. Production-grade fault tolerance usually needs a true durable-execution runtime, not just snapshots.

8. Headline numbers worth citing (Anthropic multi-agent system)

Claim	Number
Multi-agent (Opus 4 lead + Sonnet 4 subagents) vs single-agent Opus 4 on internal research evals	+90.2% (esp. breadth-first / parallel queries)
Share of performance variance explained by token usage alone on BrowseComp	~ 80% (model choice + tool-call count add ~15%)
Token cost of multi-agent vs chat	~ 15× (and ~4× a standard chat turn) → reserve for high-value tasks
Research-time reduction from parallel tool calling on complex queries	up to 90% (minutes vs hours)

The takeaway: multi-agent buys large quality gains on parallelizable, breadth-first work, but at a steep token cost — which is exactly why the reliability, eval, and observability discipline in this doc matters. You’re spending 15× the tokens; you’d better be able to tell whether it worked.

Sources

- Anthropic — How we built our multi-agent research system — <https://www.anthropic.com/engineering/built-multi-agent-research-system>
- Why Do Multi-Agent LLM Systems Fail? (MAST, arXiv 2503.13657) — <https://arxiv.org/abs/2503.13657>
· HTML: <https://arxiv.org/html/2503.13657v1>
- Where LLM Agents Fail and How They Can Learn From Failures (arXiv 2509.25370) — <https://arxiv.org/pdf/2509.25370>
- Characterizing Faults in Agentic AI: A Taxonomy — <https://arxiv.org/html/2603.06847v1>
- Looking Forward: Challenges & Opportunities in Agentic AI Reliability (arXiv 2511.11921) — <https://arxiv.org/pdf/2511.11921>
- The Compounding Errors Problem (Zartis) — <https://www.zartis.com/the-compounding-errors-problem-why-multi-agent-systems-fail-and-the-architecture-that-fixes-it/>
- ToolScan: Benchmark for Errors in Tool-Use LLMs (arXiv 2411.13547) — <https://arxiv.org/pdf/2411.13547>
- Reducing Tool Hallucination via Reliability Alignment (arXiv 2412.04141) — <https://arxiv.org/pdf/2412.04141>
- The unauthorized tool call problem (Answer.AI) — <https://www.answer.ai/posts/2026-01-20-toolcalling.html>
- AI Agent Hallucinations: Causes, Types, Tool Errors — <https://manveerc.substack.com/p/ai-agent-hallucinations-prevention>
- LangChain — LLM Evaluation Framework: Trajectories vs Outputs — <https://www.langchain.com/resources/llm-evaluation-framework>
- When AIs Judge AIs: Agent-as-a-Judge (arXiv 2508.02994) — <https://arxiv.org/pdf/2508.02994>
- Catching One in Five: LLM-as-Judge Blind Spots (arXiv 2606.10315) — <https://arxiv.org/html/2606.10315>
- Confident AI — LLM Agent Evaluation Metrics 2026 — <https://www.confident-ai.com/blog/llm-agent-evaluation-complete-guide>
- OpenTelemetry — GenAI agent & framework spans (semconv) — <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/>
- OpenTelemetry blog — Inside the LLM Call: GenAI Observability with OTel — <https://opentelemetry.io/blog/2026/genai-observability/>
- OTel GenAI agentic systems semconv issue #2664 — <https://github.com/open-telemetry/semantic-conventions/issues/2664>
- Datadog — native OTel GenAI semantic conventions — <https://www.datadoghq.com/blog/llm-otel-semantic-convention/>
- Greptime — How OpenTelemetry Traces LLM Calls, Agent Reasoning, MCP Tools — <https://greptime.com/blogs/2026-05-09-opentelemetry-genai-semantic-conventions>
- Microsoft Learn — Agent tracing in Microsoft Foundry — <https://learn.microsoft.com/en-us/azure/foundry/observability/concepts/trace-agent-concept>

- Latitude — Best AI Agent Observability Tools 2026 — <https://latitude.so/blog/best-ai-agent-observability-tools-2026-comparison>
- Langfuse — Phoenix/Arize alternatives comparison — <https://langfuse.com/faq/all/best-phoenix-arize-alternatives>
- digitalapplied — Agent Observability: LangSmith, Langfuse, Arize 2026 — <https://www.digitalapplied.com/blog/agent-observability-platforms-langsmith-langfuse-arize-2026>
- Pydantic — AI Observability Pricing Compared — <https://pydantic.dev/articles/ai-observability-pricing-comparison>
- OpenAI — A Practical Guide to Building Agents (PDF) — <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>
- OpenAI — Guardrails and human review — <https://developers.openai.com/api/docs/guides/agents/guardrails-approvals>
- OpenAI — Safety in building agents — <https://developers.openai.com/api/docs/guides/agent-builder-safety/>
- Domino.ai — Composable guardrails for agentic AI — <https://domino.ai/resources/blueprints/composable-guardrails-for-agentic-ai>
- ToolHalla — AI Agent Guardrails & Output Validation 2026 — <https://toolhalla.ai/blog/ai-agent-guardrails-io-validation-2026>
- Thinking Machines Lab — Defeating Nondeterminism in LLM Inference — <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>
- Understanding/Mitigating Numerical Nondeterminism in LLM Inference (arXiv 2506.09501) — <https://arxiv.org/pdf/2506.09501>
- LLM Determinism in Prod: Temperature, Seeds, Replayable Results — <https://medium.com/@2nick2patel2/llm-determinism-in-prod-temperature-seeds-and-replayable-results-8f3797583eb1>
- Trust at Scale: Regression Testing Multi-Agent Systems — <https://bhargavaparv.medium.com/trust-at-scale-regression-testing-multi-agent-systems-in-continuous-deployment-environments-99dfcc5872e9>
- LangGraph — Durable execution docs — <https://docs.langchain.com/oss/python/langgraph/durable-execution>
- LangGraph vs Temporal for AI Agents (durable execution) — <https://medium.com/data-science-collective/langgraph-vs-temporal-for-ai-agents-durable-execution-architecture-beyond-for-loops-a1f640d35f02>
- Diagrid — Checkpoints Are Not Durable Execution — <https://www.diagrid.io/blog/checkpoints-are-not-durable-execution-why-langgraph-crewai-google-adk-and-others-fall-short-for-production-agent-workflows>
- Zylos Research — Durable Execution Patterns for AI Agents — <https://zylos.ai/research/2026-02-17-durable-execution-ai-agents>

Cost, Performance & Economics of Multi-Agent Systems

Source: [readme/research/agent-orchestration/AGENT_ORCHESTRATION_07_COST_PERFORMANCE_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is about how much it costs to run AI systems that use many “agents” (AI workers) at the same time, and how to keep that cost under control. It matters because running several AI agents in parallel can get expensive very fast — understanding when it is worth the money, and how to cut the bill, is the difference between a useful tool and a runaway expense.

The main parts explained simply:

- **The basic price gap** — A single AI chat costs, say, \$1. One AI agent doing a task costs about \$4. A team of agents working together costs about \$15. That 15× jump is the central fact the whole document builds on.
- **When it is actually worth it** — Using many agents pays off only for big, high-value jobs that can be broken into independent pieces running at the same time: things like deep market research, legal checks, or scanning hundreds of documents. It is NOT worth it for everyday coding tasks or anything where the steps depend tightly on each other.
- **When one agent wins** — Studies show that a single agent beats a team of agents when the job requires following a chain of connected reasoning steps and the budget is fixed. More agents do not automatically mean better answers; they just spend more.
- **Speed vs. cost trade-off** — Running agents in parallel makes the job finish faster, but costs more money. Running them one after another is cheaper but slower. Deep chains of agents add serious delays before any real work even starts.
- **Using cheaper models for routine steps** — The smartest setup is to use a powerful (expensive) model to plan and check the work, and cheaper models to do the repetitive sub-tasks. Upgrading to a better model often helps more than simply buying more tokens on a weaker model.
- **Prompt caching — the single biggest money-saver** — If many agents share the same starting instructions or shared document, you only pay full price the first time. Every agent after that reads from a stored copy at about 10% of the normal cost — a 90% discount. For a team of 50 agents this is enormous.
- **Batch processing — 50% off for non-urgent work** — If a job does not need an instant answer (overnight research sweeps, bulk analysis), you can queue it as a “batch” and get half off the price on every word in and out.
- **Rate limits** — The AI provider only allows a certain number of requests and tokens per minute. A large team of agents can hit this ceiling fast. The fix is to match how many agents you run at once to your allowed limit, use caching to reduce what counts against that limit, and back off gracefully when you get a “too many requests” error.
- **Cost-control guardrails** — You should set hard limits before any run: maximum number of agents, maximum steps each agent can take, and a maximum dollar amount per task. Without these, a stuck agent can loop forever and burn through a huge budget. Stopping early when results stop improving can cut costs by roughly 24% with no drop in quality.
- **Pass small references, not huge outputs** — Agents should hand off a short pointer or summary to the next agent, not copy the entire document. Copying big outputs through every handoff wastes tokens and slows everything down.

What to do with this:

Start with a single agent for any new task, and only move to a team of agents when the job is genuinely too large for one and the value of the output clearly justifies the roughly 15× higher cost. Always turn on prompt caching for any setup where agents share the same instructions, and set a hard dollar cap on every run before you start.

When is fanning out across many agents worth the token bill — and how do you stop it running away?

Research/reference doc · 2026-06-16 · part of the Agent Orchestration suite

This is part 07 of an 8-part suite. It focuses on the **money and latency** of multi-agent systems: what they cost, when the cost is justified, and the concrete levers (tiering, caching, batching, budget caps) that bring the bill down without losing quality.

1. The headline number: ~15× a chat

Anthropic's own production research-agent measurements anchor the whole discussion. On the same scale:

chat (1×) → single agent (~4×) → multi-agent (~15×)
tool use & loops parallel fan-out + per-subagent context

- “Multi-agent systems use about **15× more tokens** than chats.” (Anthropic)
- “Agents typically use about **4× more tokens** than chat interactions.” (Anthropic)

So most of the multiplier is *spending more total tokens on purpose* — every subagent runs its own context window, its own tool calls, and its output is (often) reprocessed by the orchestrator.

What actually drives performance

In Anthropic's eval, **three factors explained 95% of performance variance**:

Factor	Share of variance
Total token usage	~80%
Number of tool calls	~10%
Model choice	~5%

Takeaway: outcomes are dominated by *how much thinking/searching you spend*, not by clever topology. Multi-agent architecture is mostly a way to **spend more tokens productively in parallel** — it does not conjure quality from a fixed budget.

The economic gate

“These architectures burn through tokens fast. For economic viability, multi-agent systems require tasks where the value of the task is high enough to pay for the increased performance.” — Anthropic

Token cost is typically the single largest line item in a production agentic system. An unconstrained agent can cost **\$5–\$8 per task** to solve a single software-engineering issue (Stevens). A single agentic task can fan out to hundreds of model calls and exceed 1M tokens.

Verification flag: Third-party overhead estimates ($\approx 58\%$ overhead for decentralized, $\approx 285\%$ for centralized supervisors that reprocess everything) are illustrative secondary-source figures, not Anthropic measurements. Use them for intuition only.

2. When multi-agent is worth it — and when it's waste

Worth it

“Multi-agent systems excel at valuable tasks that involve heavy parallelization, information that exceeds single context windows, and interfacing with numerous complex tools.” — Anthropic

“Multi-agent research systems excel especially for **breadth-first queries** that involve pursuing multiple independent directions simultaneously.” — Anthropic

The repeatedly-named economic sweet spot is **high-value research**: legal due diligence, competitive/market intelligence, biomedical literature review. Each subagent's own context window effectively multiplies usable context and enables true parallel exploration.

Poor fit / waste

“Most coding tasks involve fewer truly parallelizable tasks than research, and LLM agents are not yet great at coordinating and delegating to other agents in real time.” — Anthropic

Poor fits: **coding**, real-time coordinated work, and any task where subtasks share state or depend on each other's intermediate results (handoffs lose fidelity).

Counter-evidence: single agent can win even on hard tasks

A controlled study (arXiv 2604.02460) found that **single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking-token budgets** (HotpotQA, MuSiQue). Mechanisms of

waste:

- **Redundant reasoning** — agents independently re-solve the same subproblems.
- **Coordination overhead** — orchestration messages cost tokens and add no reasoning.
- **“Context rot”** — information degrades across handoffs; a single focused agent keeps better contextual coherence on interdependent chains.

The rule this gives you: multi-agent pays off only when you’re willing to spend *more* total tokens to buy parallelism/breadth — **not** as a way to get more out of a *fixed* budget. Under a constrained budget on interdependent reasoning, consolidate into one agent.

The distractor-domain nuance (LangChain benchmark)

Condition	Single agent	Multi-agent
0–1 distractor domains	slightly better , far cheaper	unnecessary overhead
2+ distractor domains	accuracy falls off sharply	accuracy holds; token use stays flat

Single-agent token use climbs with context bloat as irrelevant domains accumulate; supervisor/swarm token use stays flat. Multi-agent’s efficiency edge appears **specifically under high irrelevant-context / many-domain conditions**.

3. Latency vs. throughput

Core tension: parallel fan-out cuts wall-clock time but raises total token cost; deep hierarchies and reflection loops add serial latency.

Pattern	Approx. wall-clock
Single LLM call	~800 ms
Orchestrator-worker + Reflexion loop	~10–30 s
3-level hierarchy (2 s/level)	≥6 s before any worker even starts

SERIAL (latency stacks)

```
graph TD
  orchestrator --> subagent
  subagent --> sub_subagent
  sub_subagent --> end
```

critical path = SUM of stages

PARALLEL (latency = longest path)

```
graph TD
  orchestrator --> subagent_A[subagent A]
  orchestrator --> subagent_B[subagent B]
  orchestrator --> subagent_C[subagent C]
  subagent_A --> synthesize
  subagent_B --> synthesize
  subagent_C --> synthesize
```

critical path = MAX single path

- Anthropic’s lead agent “spins up **3–5 subagents in parallel** rather than serially; the subagents use **3+ tools in parallel**. These changes **cut research time by up to 90%** for complex queries.”

- LangGraph framing: parallelism reduces critical-path latency from the *sum* of agent times to the *max* path through the dependency graph.
- **Compounding cost warning:** “a Reflexion loop that runs for 10 cycles can consume **50× the tokens** of a single linear pass” (Stevens).

Rule of thumb: parallel fan-out trades money for wall-clock time — use it when latency matters and the task is genuinely parallel. Avoid deep serial hierarchies and unbounded reflection loops when cost matters.

4. Model tiering — strong orchestrator, cheap workers

Anthropic’s production config: **Claude Opus** as lead/orchestrator + **Claude Sonnet** subagents.

“Multi-agent system with Claude Opus 4 as the lead agent and Claude Sonnet 4 subagents **outperformed single-agent Claude Opus 4 by 90.2%** on our internal research eval.” — Anthropic

And a key spending heuristic:

“Upgrading to Claude Sonnet 4 is a larger performance gain than doubling the token budget on Claude Sonnet 3.7.” — Anthropic

General principle: route smaller / specialized / high-volume subtasks to cheaper models (Haiku, open-source), reserve the frontier model for orchestration and final synthesis. **Spend on a better model before spending on more tokens.**

Reference pricing (per million tokens, standard sync rates)

Model	Input	Output
Opus (4.x)	\$5.00	\$25.00
Sonnet (4.x)	\$3.00	\$15.00
Haiku (4.5)	\$1.00	\$5.00

Verification flag: Exact per-model prices are time-sensitive — confirm against Anthropic’s live pricing page before relying on them. The durable point is the **ratio**: Haiku input is $\approx 1/5$ the cost of Opus input. That ratio is what makes worker-tiering pay.

5. Prompt caching & batching — the biggest levers

Prompt caching

Operation	Multiplier vs. standard input	Effect
Cache write (5-min TTL)	1.25×	small premium to write
Cache write (1-hour TTL)	2.0×	larger premium, longer life
Cache read / hit	0.10×	90% discount

Applied to Sonnet (\$3 base input): write costs \$3.75 (5-min) or \$6.00 (1-hr); a **hit costs \$0.30**. Caching reduces input cost ~90% and latency ~75%.

Why it's the #1 lever in fan-out systems:

- An orchestrator spawning ~50 workers that share the same prefix (system prompt, task brief, tool definitions, shared corpus) pays full price once and ~10% thereafter — caching “effectively eliminates the redundancy penalty.”
- **Cached tokens do not count toward ITPM rate limits** → 5–10× effective throughput. This rate-limit exemption is often more valuable than the cost saving itself.
- Anthropic calls prompt caching “the single highest-ROI optimization available” for reused-context workloads. Example RAG pipeline: **\$150/day** → **~\$22/day (~85%)**.

Batch API (Message Batches)

- **50% discount on both input AND output**, all Claude models, in exchange for async completion (up to 24h).
- Sonnet batched: **\$1.50 input / \$7.50 output**.
- Use for latency-tolerant work: overnight research sweeps, bulk subagent jobs. **Not** for interactive orchestration.

Stacking

Caching (~90% input) and batch (~50%) reportedly compound — a corpus/RAG pipeline can bill at **~5% of base rate card** (~95% total reduction).

Verification flag: Stacking to ~5% of rate card is asserted by secondary sources (codewords / aicostcheck), **not** confirmed on Anthropic’s own pricing page. Verify before stating as fact in customer-facing material.

6. Concurrency & rate limits

Anthropic enforces **three dimensions per model class**, via a **token-bucket** algorithm (continuous replenishment, not a fixed-window reset):

- **RPM** — requests per minute
- **ITPM** — input tokens per minute
- **OTPM** — output tokens per minute

Usage tiers (deposit-gated; approximate)

Tier	Entry	~RPM
Free	—	~5
1	\$5	~50
2	—	~1,000
3	—	~2,000
4	\$400+	~4,000

Tier 4 is also cited around ~2,000,000 ITPM / ~400,000 OTPM for Sonnet/Opus 4.x.

Verification flag: All tier RPM/TPM numbers are time-sensitive — check Anthropic’s live rate-limits doc.

Error handling

- **429** `rate_limit_error` = a tier limit was exceeded; the response carries a `retry-after` header (seconds).
- **529** = service overload (different cause; back off and retry).

Multi-agent guidance: fan-out hammers rate limits faster than anything else. When throttled:

1. **Reduce client-side concurrency** so the next minute doesn’t also fail.
2. If you keep hitting it, “you have a capacity problem that retries will not fix — upgrade tier, enable prompt caching, or shed load.”

Rule of thumb: size max-concurrent-subagents to your tier’s **ITPM/OTPM, not just RPM**; add exponential backoff + jitter on 429; cap fan-out width; cache the shared prefix to lift effective throughput.

7. Cost-control patterns

A. Scale effort to query complexity (prompt the budget in)

Agents can't reliably self-judge effort, so the budget must be **prompted in**. Anthropic's embedded scaling rules:

Query type	Agents	Tool calls
Simple fact-finding	1	3–10 total
Direct comparison	2–4 subagents	10–15 each
Complex research	10+ subagents	divided responsibilities

Purpose: “prevent overinvestment in simple queries, which was a common failure mode in our early versions.”

B. Runtime budget guardrails

```
per-call token count → running total → enforce hard limits
├─ warn at threshold
└─ reject above limit / dollar cap
```

Given execution state + observed usage + expected remaining work, decide: **continue** · **narrow** · **degrade** · **reroute** · **escalate** · **stop**. Detect **low-yield loops** (repeated retrieval retries, premium-model calls that don't reduce uncertainty) and cut that path's budget. Set caps at **agent / workflow / org** levels with hard **dollar caps** per task/run.

C. Dynamic turn limits / early exit

“Dynamic turn limits based on the probability of success can cut costs by **24%** while maintaining solve rates” (Stevens) — stop iterating once the marginal confidence gain flattens.

D. Loop-until-dry, but bounded

Let agents work until no new useful information appears (“dry”) — but always bound by **max iterations** + **token/dollar cap** + **timeout**, so a stuck loop can't trigger the 50×-tokens reflection-loop blowup.

E. Pass references, not payloads

Subagents “store their work in external systems, then pass lightweight references back to the coordinator.” — Anthropic

Avoid copying large outputs through conversation history. Summarize completed phases into external memory before continuing when context fills. Retrieval from cached plans can drop latency “from 30 seconds to 300 ms and reduce cost to near zero.”

F. Architecture-level savings (LangChain benchmark)

Removing handoff messages and using **direct message/tool forwarding** — letting a worker's output pass through verbatim instead of the supervisor regenerating/paraphrasing it — gave a **~50% performance increase** and cut the supervisor's "translation" token tax. The supervisor pattern is the most general but consistently costs more than a swarm unless you apply these forwarding optimizations.

8. Rules of thumb (consolidated)

1. **Default to a single agent.** Reach for multi-agent only for high-value, breadth-first, parallelizable tasks that exceed one context window or need many tools.
 2. **Don't use multi-agent to stretch a fixed budget** on interdependent reasoning — under equal budgets a single agent wins (context rot kills handoffs).
 3. **Spend follows value:** budget $\approx 15\times$ a chat. Only justified when outcome value clears that bar.
 4. **Tier models:** strong orchestrator + cheap workers; upgrading the model can beat doubling tokens.
 5. **Cache the shared prefix** (system prompt, brief, tool defs, corpus) — 90% input discount, ~75% latency cut, and exempt from ITPM. Highest-ROI lever in fan-out.
 6. **Batch latency-tolerant work** for another 50% off; caching + batch reportedly compound.
 7. **Bound everything:** max subagents, max tool calls, max turns/iterations, per-task dollar cap, timeout.
 8. **Match concurrency to ITPM/OTPM, not RPM;** backoff + jitter on 429; cache to lift effective throughput.
 9. **Pass references, not payloads;** forward worker output verbatim; summarize to external memory.
 10. **Early-exit on diminishing returns** (dynamic turn limits $\approx -24\%$ cost at equal solve rate).
-

9. Cost/benefit decision checklist

Use multi-agent when *most* are true:

- [] Task value absorbs $\sim 15\times$ chat token cost (due diligence, competitive/market intel, biomedical lit review).
- [] Work decomposes into **independent, parallel** strands (breadth-first), not a tightly coupled chain.
- [] Required information **exceeds a single context window**, or the task spans many domains / many tools.
- [] Wall-clock latency matters and parallel fan-out meaningfully shortens it.
- [] You can supply a **strong orchestrator + cheaper workers** without losing quality at synthesis.
- [] A large **reusable shared context** exists to cache (system prompt, brief, corpus, tool defs).
- [] Your tier's ITPM/OTPM can sustain the planned concurrency (or you'll cache / batch / upgrade).
- [] You can enforce **hard caps** (subagent count, tool calls, turns, dollars, timeout) and early-exit.

Prefer a single agent when:

- [] Budget is fixed and the task is **interdependent multi-hop reasoning** (single agent wins under equal budget).
- [] Task is **coding** or real-time coordinated work.
- [] There are **0–1 relevant domains** / low distractor context (single agent matches or beats, far cheaper).
- [] Outcome value can't justify the token multiple.

Sources

- Anthropic Engineering — How we built our multi-agent research system — <https://www.anthropic.com/engineering/multi-agent-research-system>
- LangChain — Benchmarking Multi-Agent Architectures — <https://www.langchain.com/blog/benchmarking-multi-agent-architectures>
- LangChain Docs — Subagents (multi-agent) — <https://docs.langchain.com/oss/python/langchain/multi-agent/subagents>
- The Hidden Economics of AI Agents: Token Costs and Latency Trade-offs (Stevens Online) — <https://online.stevens.edu/blog/hidden-economics-ai-agents-token-costs-latency/>
- Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets (arXiv) — <https://arxiv.org/pdf/2604.02460>
- Finout — Anthropic API Pricing 2026 (caching multipliers, batch, per-model rates) — <https://www.finout.io/blog/anthropic-api-pricing>
- codewords.ai — Anthropic Batch API: 50% cost — <https://www.codewords.ai/blog/anthropic-batch-api>
- AI Cost Check — Prompt Caching Savings: OpenAI vs Anthropic — <https://aicostcheck.com/blog/ai-prompt-caching-cost-savings>
- Anthropic API Docs — Rate limits — <https://docs.anthropic.com/en/api/rate-limits>
- Respan — Anthropic API Rate Limits + 429/529 Handling Guide (2026) — <https://www.respan.ai/articles/anthropic-api-rate-limits>
- DevTk.AI — AI API Rate Limits 2026 (RPM, TPM, 429) — <https://devtk.ai/en/blog/ai-api-rate-limits-comparison-2026/>
- Oracle Blogs — Runtime Budget Guardrails for Agentic AI — <https://blogs.oracle.com/ai-and-datascience/runtime-budget-guardrails-agentic-ai>
- Maisa AI — Agentic AI Cost Control — <https://maisai.ai/agentic-insights/agentic-ai-cost-control/>
- Portal26 — AI Agent Cost Control — <https://portal26.ai/ai-agent-cost-control-stop-agents-burning-budget/>
- DataRobot — Balancing cost and performance: Agentic AI development — <https://www.datarobot.com/blog/cut-agentic-ai-development-costs/>
- Introl — Prompt Caching Infrastructure (cost/latency reduction guide 2025) — <https://introl.com/blog/prompt-caching-infrastructure-llm-cost-latency-reduction-guide-2025>

- [decodethefuture — Multi-Agent Systems Explained: 2026 Patterns —](https://decodethefuture.org/en/multi-agent-systems-explained/)
<https://decodethefuture.org/en/multi-agent-systems-explained/>
- [ByteByteGo — How Anthropic Built a Multi-Agent Research System —](https://blog.bytebytego.com/p/how-anthropic-built-a-multi-agent)
<https://blog.bytebytego.com/p/how-anthropic-built-a-multi-agent>

Applied: Agent Orchestration in Print-Flow-360

Source: [readme/research/agent-orchestration/AGENT_ORCHESTRATION_08_APPLIED_PRINTFLOW360_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document explains how the AI features inside Print-Flow-360 actually work right now — and where smarter AI could be added in the future. It matters because AI tools like the product builder and design generator directly save time for store owners, and knowing what is real versus what is just an idea helps set realistic expectations for what to build next.

The main parts explained simply:

- **The 7 AI features that exist today** — The platform has seven AI tools: a product builder (you describe a product in plain words and it creates a draft), an SEO title and description writer, a product description writer, an email template writer, a design layout generator, an image generator, and a background remover. Each works on its own.
- **One shared pipeline for all AI calls** — Every feature sends its request through one central piece of code. Think of it as a shared telephone exchange. Any feature can use it; it handles sending the question to the AI provider, getting the answer back, and recording what the call cost.
- **Per-store API keys** — Each store uses its own AI provider account (Anthropic or OpenAI), stored in that store's settings. No store's credits are shared with another store.
- **Single question, single answer — no memory** — Every AI call today is a one-shot exchange: one question in, one answer out. The AI does not remember earlier questions and does not loop back to check its own work. Simple and reliable, but limited in what it can do.
- **The design generator's safety guardrails** — The design AI is the most advanced existing feature. It is allowed to pick only from a fixed list of templates, colors, and fonts — it cannot invent a free-form layout. This is deliberate: it stops the AI from producing unpredictable or unusable results. This guardrail approach is the right model to copy when adding more AI features.
- **Four things the AI cannot do yet** — It cannot call internal tools (like searching the product catalog), cannot cache large system prompts to save cost, cannot match an existing catalog product (it only creates new ones), and all AI runs live inside the web request — meaning multi-step AI would slow the page down.
- **Future ideas mapped out (none built yet)** — The document outlines four realistic upgrades: a smarter product matcher that finds an existing product instead of always creating a new one; a writing chain that generates a description, then SEO text, then a launch email in one flow; an automatic customer support sorter; and a design quality checker that regenerates a design if it fails basic checks. All are research only.

What to do with this: The single most important step before any multi-step AI idea can work is enabling background job processing (switching from `sync` queue to Redis). Without it, any AI flow that needs more than one step will freeze the page for the user while it waits. Fix that first, then any of the future opportunities become buildable.

How the orchestration patterns from this suite map onto the AI that actually ships in this codebase — and the realistic seams where multi-agent work could land.

Research/reference doc · 2026-06-16 · part of the Agent Orchestration suite

0. Read this first

This is the **applied** chapter. Parts 01–07 covered the general theory (single-agent loops, orchestrator-worker, prompt chaining, evaluator-optimizer, routing, tool-use, evaluation). This part grounds all of that in `app/Services/AI/` as it exists today.

Two rules govern everything below:

1. **Code is ground truth, docs are intent.** `readme/AI_PRODUCT_BUILDER.md` and `readme/AI_PRODUCT_MODULE_CONTEXT.md` describe a *catalog-matcher* flow. The shipped code does **not** match a catalog — it *creates* products. Where doc and code disagree, the code wins and this doc says so.
2. **Nothing in §4 is built.** The opportunity mapping is research-only. No orchestration work has been done.

1. What AI ships today

All AI lives under `app/Services/AI/`. There are **7 AI/LLM consumers**, every one routed through a single shared runner. (The `AI_FEATURE_SOP.md` “registry” table is stale — it lists only `product_builder` and `seo_meta`; the code has five more.)

Feature slug	Service chain	What it does	Backend
<code>product_builder</code>	<code>IntentParserService</code> → <code>AiProductBuilderOrchestrator</code> → <code>AiProductCreatorService</code>	NL prompt (“black hoodie”) → full draft <code>Product</code> (sizes, options, pricing rule) saved as <code>INACTIVE</code> draft + designer URL	LLM JSON
<code>seo_meta</code>	<code>SeoMetaGeneratorService</code> (<code>SeoMetaContext</code> , <code>SeoMetaDto</code>)	Product name/desc → meta title, description, keywords	LLM JSON
<code>product_description</code>	<code>ProductDescriptionGeneratorService</code>	Generate / rewrite product copy	LLM JSON
<code>email_template</code>	<code>EmailTemplateGeneratorService</code>	Generate / improve HTML email body, preserves <code>{merge_tags}</code>	LLM raw text (<code>expectsJson:false</code>)
<code>designer_image_gen</code>	<code>DesignerImageGeneratorService</code> (<code>DesignerImageContext</code>)	LLM optimizes prompt → DALL-E / <code>gpt-image-1</code> → tenant S3	LLM + OpenAI image API
<code>designer_design</code>	<code>DesignGeneratorService</code> (<code>DesignContext</code>)	LLM as “design director”: picks layout <i>template</i> per page + words/colors/fonts; geometry owned by designer’s <code>utils/aiDesignMapper.ts</code>	LLM JSON, <code>maxTokens:4000</code>
<code>background_removal</code>	<code>BackgroundRemovalService</code>	<code>remove.bg</code> API (not an LLM, but logged through the AI usage spine)	<code>remove.bg</code>

Routes split by audience:

- **Admin AI** (`routes/api.php:678-687`): `/ai/status` , `/ai/product-builder` , `/ai/seo-meta` , `/ai/product-description` , `/ai/email-template` , `/ai/usage*` .
- **Designer/studio AI** (`routes/store-api.php:403-406` → `DesignerAiController`): `status` , `generate-image` , `generate-design` , `remove-background` .

What does NOT exist today: no support-triage AI, no chat/conversational agent, no content/blog pipeline. (Grep for `ticket` / `support` in `app/Services/AI` only hits the unrelated `Support/` JSON-helper namespace and code comments.)

2. The real architecture: one stateless runner, thin feature layers

The design is a clean **thin feature layer over a feature-agnostic runner**. Every call is **single-shot, single-turn, stateless** — one `system` prompt + one `user` message in, one completion out.

```

Controller (FormRequest → ... → successResponse)
|
▼
Per-feature Service / Orchestrator
|  builds AiTaskRequest { feature, Context, maxTokens, expectsJson }
|
▼
AiTaskRunner::run(AiTaskRequest): AiTaskResponse
|  1. LlmManager resolves provider (per-tenant key)
|  2. provider->complete($system, $user, $maxTokens)
|  3. optional JsonExtractor::extract()
|  4. write EXACTLY ONE ai_usage_logs row
|
▼
AiTaskResponse → orchestrator patches domain outcome
                  onto the SAME log row (forceFill->save)

```

The components

- **AiTaskRunner::run()** (`app/Services/AI/AiTaskRunner.php`) — the single entry point. Resolves provider, calls `complete()`, optionally extracts JSON, and writes **exactly one** `ai_usage_logs` row (feature, provider, model, tokens, latency, confidence, status). Domain outcomes are *patched onto that same row* by each orchestrator rather than logged as new rows.
- **LlmManager** (`app/Services/AI/LlmManager.php`) — resolves the active provider from the **tenant's** `store_third_party_services` record via `ThirdPartyServiceFetcher::resolveAny(['anthropic','openai'], $storeId, activeOnly:true)`. **API keys are per-tenant, never .env**. Honors an `enable_debug_mode` flag that swaps in an inline **mock provider** returning canned product JSON (no credits spent). Throws plain-language “Go to Settings → Integrations” errors when unconfigured.
- **Providers** (`AnthropicProvider`, `OpenAiProvider`, `AbstractLlmProvider`) — minimal HTTP clients implementing `LlmProviderInterface::complete()`. Anthropic uses `claude-sonnet-4-6` (`config/ai.php`). Both send only `system` + a single `user` message.
- **Per-feature layer** — a `Context` (prompt builder: `getSystemPrompt()` + `buildUserMessage()`) + a `Dto` (lenient `fromArray()` that never throws on partial JSON) + an optional orchestrator + a thin controller + a Nuxt composable.

Four facts that constrain orchestration (verified in code)

#	Constraint	Evidence	Why it matters for orchestration
1	No native tool-use / function calling	grep <code>tools tool_use tool_choi</code> <code>ce function_call</code> in <code>app/Services/AI</code> = 0 hits	All “structured output” is prompt-and-parse JSON via <code>JsonExtractor</code> . Any orchestrator step that needs to call an internal function (catalog search, pricing calc) has no clean mechanism today.
2	No prompt caching	grep <code>cache_control</code> = 0 hits	Static system prompts (e.g. <code>ProductSystemContext</code> , ~146 lines of print-domain knowledge) are re-sent verbatim every call. Chained/looped flows pay full cost each step.
3	No live-catalog injection	<code>ProductSystemContext</code> is static domain knowledge only	The product <i>builder</i> invents a brand-new product; it does not retrieve-then-match. The documented <code>ProductMatcherService</code> is aspirational.
4	<code>QUEUE_CONNECTION=sync</code>	<code>config/queue.php:16</code> + <code>.env</code>	All AI runs inline in the HTTP request . Any multi-step or fan-out flow would block the request. This is the single biggest gating fact.

The most “agent-like” thing today

`DesignGeneratorService` is the closest existing precedent to a constrained agent: the LLM is boxed into an **allowlist** (`ALLOWED_TEMPLATES`, `ALLOWED_CONTENT`, palette/font validation) and is **never trusted for geometry** — layout math is owned by `aiDesignMapper.ts`. This “LLM proposes within a fixed vocabulary, host validates and renders” split is the guardrail pattern the rest of the system should copy before adding autonomy.

By contrast, the `AiProductBuilderOrchestrator` is *not* a multi-agent or LLM-loop orchestrator — it is a **sequential pipeline** (parse → create → patch log). Useful seam, but don’t mistake the class name for genuine orchestration.

3. Mapping the suite’s patterns onto this codebase

Pattern (from parts 01–07)	Does it exist here?	Closest existing code
Single-shot completion	✓ Everywhere	<code>AiTaskRunner::run()</code>
Sequential pipeline	✓ (one case)	<code>AiProductBuilderOrchestrator</code> (parse → create → log)
Constrained “propose within vocabulary”	✓ (one case)	<code>DesignGeneratorService</code> allowlists
Prompt chaining (output → input across calls)	✗	—
Orchestrator-worker (fan-out to specialists)	✗	—
LLM-as-router	✗	<code>IntentParserService</code> is the nearest seam
Evaluator-optimizer (generate → critique → retry)	✗	<code>sanitizePage</code> / <code>cleanPalette</code> are a mechanical, non-LLM precursor
Native tool-use	✗	blocked by constraint #1

4. Where orchestration could realistically help

Honest opportunities mapped to existing code. **None of this is built.** Each lists its real prerequisite.

4.1 Product Builder → true orchestrator-worker

Today: `product_builder` is one LLM call that *invents* a product.

Opportunity: the documented catalog-match design maps cleanly to **orchestrator-worker**. A lead step decides intent, then dispatches specialists:

```
IntentParser (lead)
├─ catalog retrieval/match worker ← the MISSING ProductMatcherService
├─ pricing-rule worker
└─ SEO + description workers ← already exist as services
```

The existing per-feature services are already composable, and `AiProductBuilderOrchestrator.php` is the natural fan-out seam. **Best fit for native tool-use:** let the model call a `search_catalog` tool instead of prompt-stuffing the catalog.

When to use: once stores have real catalogs and “match my existing product” beats “invent a new one.” **When NOT to:** for the cold-start / empty-catalog tenant, invent-a-product is correct — don’t

force a matcher with nothing to match. **Prereq:** add tool-use to `AnthropicProvider::complete()` (currently text-only) + a multi-step request model + async (constraint #4).

4.2 Content/SEO chaining (chaining, not multi-agent)

Today: `SeoMetaGeneratorService`, `ProductDescriptionGeneratorService`, `EmailTemplateGeneratorService` are independent one-shot calls.

Opportunity: a **prompt-chaining pipeline** — *generate description* → *derive SEO meta from it* → *draft a launch email referencing it* — reuses all three under a thin coordinator. Low risk, high value for store-owner onboarding; could also drive the not-yet-existent CMS-block/blog content.

Prereq: a 3-call chain is a multi-second blocking request under `QUEUE=sync`. Make it a queued job once Redis lands.

4.3 Support triage (greenfield — strongest “new agent” case)

Today: no support/ticket spine and no support AI (consistent with the known “no support/ticket spine” launch blocker).

Opportunity: a triage classifier/router is the textbook single-purpose agent — classify inbound message → route / tag / draft reply. It plugs into the existing spine trivially: a new `feature: 'support_triage'`, a `SupportTriageContext`, a DTO, then `AITaskRunner::run(...)`. The cleanest place to introduce an **LLM-as-router** without touching anything fragile.

Prereq: a support/ticket data model must exist first. The AI is the easy half.

4.4 Design-studio assist → evaluator-optimizer loop

Today: `DesignGeneratorService` constrains the model to templates and validates output, but does not loop.

Opportunity: an **evaluator-optimizer loop** — generate a design spec → score it (contrast / safe-margin / hierarchy checks, partly mechanizable like the JS contrast audits from the Aurora QA work) → regenerate if it fails. The existing sanitization layer (`sanitizePage`, `cleanPalette`, `cleanColor`) is the natural seam for an automated evaluator.

Prereq: looping = multiple LLM calls = must be async (Redis). Also needs a defined acceptance rubric so the evaluator isn't itself a coin-flip.

4.5 Cross-cutting enablers (every pattern above depends on these)

Enabler	Current state	Unblocks
Async execution (flip <code>QUEUE_CONNECTION</code> off <code>sync</code> → Redis)	<code>sync</code>	#1 prerequisite for <i>every</i> >1-call flow; today they'd block the request
Native tool-use in <code>AnthropicProvider</code> (+ OpenAI function calling)	absent	orchestrator steps that must call internal functions (catalog search, pricing, inventory)
Prompt caching on large static system prompts	absent	cuts cost/latency of chained/looped flows that re-send <code>ProductSystemContext</code> etc. verbatim
<code>ai_usage_logs</code> observability spine	✅ already exists	already a real asset — keep the “one row, patch outcomes” discipline rather than logging per sub-step

5. Honest caveats

- **All of §4 is research-only.** Nothing was implemented.
- **Image gen is stubbed.** `DesignerImageGeneratorService::callDalle()` exists, but `generate()` currently returns a `placeholder.co` placeholder (line 52). Wired, not shipped — flag this before citing it as live.
- **Matcher docs ≠ shipped code.** `AI_PRODUCT_BUILDER.md` / `AI_PRODUCT_MODULE_CONTEXT.md` describe a catalog *matcher*; the code *creates* products. Treat those docs as design intent; treat `app/Services/AI/*` as ground truth.
- **The one observability discipline to keep:** an orchestrator should still write one `ai_usage_logs` row and patch outcomes onto it (`AiProductBuilderOrchestrator` is the canonical example), not fan out into a row per sub-step — otherwise per-tenant cost/quality tracking fragments.

6. Key file citations

Area	Files
Runner / manager / request	<code>app/Services/AI/AiTaskRunner.php</code> , <code>LlmManager.php</code> , <code>AiTaskRequest.php</code>
Providers / config	<code>app/Services/AI/Providers/{AnthropicProvider,OpenAiProvider,AbstractLlmProvider}.php</code> , <code>config/ai.php</code>
Consumers	<code>app/Services/AI/{IntentParserService,AiProductBuilderOrchestrator,AiProductCreatorService,SeoMetaGeneratorService,ProductDescriptionGeneratorService,EmailTemplateGeneratorService,DesignGeneratorService,DesignerImageGeneratorService,BackgroundRemovalService}.php</code>
Contexts / DTOs	<code>app/Services/AI/Context/*</code> , <code>app/DTO/AI/*</code>
Routes	<code>routes/api.php:678-687</code> , <code>routes/store-api.php:403-406</code>
Queue gating	<code>config/queue.php:16</code> + <code>.env</code> <code>QUEUE_CONNECTION=sync</code>
Docs (intent, not ground truth)	<code>readme/AI_FEATURE_SOP.md</code> , <code>readme/AI_PRODUCT_BUILDER.md</code> , <code>readme/AI_PRODUCT_MODULE_CONTEXT.md</code>

Sources

- This part is grounded in the Print-Flow-360 codebase itself (files cited in §6) rather than external URLs. The orchestration-pattern vocabulary it maps onto comes from the rest of this suite, which draws on:
- Anthropic — Building effective agents — <https://www.anthropic.com/engineering/building-effective-agents>
 - Anthropic — Tool use (function calling) overview — <https://docs.anthropic.com/en/docs/build-with-claude/tool-use/overview>
 - Anthropic — Prompt caching — <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>